

论文标题

马文杰

李传浩

张可欣

摘要

随着人工智能算力规模的持续扩张，“算电协同”成为了算力负荷与电力系统实时耦合优化的现实需求。本文基于附件数据，针对多集群异构算力调度中的算电协同优化问题，遵循“需求分析—短期预测—基础调度—碳感知—储能协同—多区域协同”的递进思路，构建了四套衔接的确定性优化模型体系。

针对问题一，首先对 50000 个任务按区域、任务类型进行 GPU 需求统计分析，刻画了“东部任务多、单机需求小；西部承接训练、单机需求大”的分布特征；然后构建短期预测模型，对比 ETS、ARIMA、CEEMDAN 分解-重构与组合预测等策略，发现逐时需求序列近似“均值 + 波动”结构，ETS/ARIMA 退化为水平模型，测试集 MAPE 为 23.39%；最后建立以任务开工延迟与迁移惩罚最小化为目标的基础算力调度模型，第 2376~2399 小时到达的 538 个任务全部按时完成，区域 E、F 的 GPU 利用率达 0.718、0.794，东部区域存在较大可迁移算力空间。

针对问题二，构建以“购电成本 + 碳权重 $\times$ 碳排放 + 时延惩罚”为目标的碳感知任务调度混合整数规划模型，针对 5 万任务规模采用模拟退火算法求解。调度方案共迁移 2529 个可延迟任务至低电价、低碳区域，购电成本降低 1312.9 万元（0.60%）、碳排放降低 5639 吨（0.27%），验证了“空间迁移 + 时间迁移”策略在满足时延与容量约束下的降本减排有效性。

针对问题三，建立含荷电状态递推、充放互斥与购售电边界的储能协同优化混合整数线性规划模型，采用 Gurobi 精确求解。结果表明：配置储能后各区域负荷波动显著下降，区域 F 峰值净购电由 280.9MW 降至 268.2MW，储能的核心价值体现为削峰填谷与平抑波动，而峰谷套利空间受电价一负荷峰谷不同步限制较为有限。

针对问题四，建立多区域“算—储—电”协同优化的统一混合整数规划模型，将任务迁移、储能充放电与购售电联合决策，并对比基准、碳约束、电价机制与新能源波动四类场景。结果表明：碳约束场景引导低碳迁移，碳排放降低 15655 吨（0.77%）；新能源出力降低 30% 场景下购电成本上升 13.3%、碳排放上升 11.6%。灵敏度分析进一步表明，碳价存在约 300 元/吨的“经济—低碳”双赢阈值，储能容量配置已达边际收益饱和区间，而新能源出力波动对系统影响最为显著。

本文的创新性在于：其一，构建了“需求分析—短期预测—基础调度—碳感知—储能协同—多区域协同”的递进式模型体系，前三问结果与问题四统一衔接；其二，针对运行成本、碳排放、网络时延、服务质量、新能源利用率、峰值净购电与负荷波动等多类量纲不同的指标，建立了统一的多目标加权优化框架，并针对不同问题规模分别采用模拟退火与精确求解等匹配求解策略；其三，将任务迁移、储能充放电与购售电纳入统一的“算—储—电”协同优化模型，并通过碳约束、电价机制与新能源波动等多场景对比及灵敏度分析，验证了模型结论的稳健性。

关键词：算电协同；混合整数规划；模拟退火；储能优化；碳感知调度；灵敏度分析

1 问题重述

随着人工智能算力规模的持续扩张，数据中心的用电需求、碳排放及其与电网的交互影响不断增强，电力成本已成为数据中心运营成本的核心组成部分。在此背景下，“算电协同”成为了算力负荷与电力系统实时耦合优化的现实需求。由于东部地区时延低、实时业务需求高，而西部地区具备电价低、新能源丰富的优势，跨区域算力迁移与能源调度构成一个多目标协同优化问题。赛题设定六个典型区域：区域 A、区域 B、区域 C 为东部高负荷区域，区域 D 为西部算力中心，区域 E、区域 F 分别为光伏与风电资源富集区域。任务到达主时域为第 0-2399 小时，第 2400-2405 小时用于结清末端任务，第 2406 小时不安排任务，仅进行电力与储能终端状态结算。任务分为实时推理、批量推理与人工智能训练三类，均不可抢占、不可拆分或中途迁移。赛题要求基于附件提供的 GPU 容量、任务轨迹、网络时延、电价、碳强度、新能源出力与储能等数据，依次解决以下四个问题：

问题一是算力需求分析与短期预测。对不同区域、不同任务类型的 GPU 需求进行统计分析并构建短期预测模型，进而建立基础算力调度模型，给出第 2376-2399 小时实际到达任务的调度方案，并展示最后 24 小时的调度甘特图与区域 GPU 利用率；问题二是碳感知任务调度。以第 0-2399 小时实际到达任务及逐时电力参数为输入，以任务迁移与开工时段为决策变量，以运行成本、碳排放、网络时延和新能源利用率为目标或评价指标，建立碳感知任务调度模型并给出调度策略；问题三是储能协同优化。在给定任务调度与逐时负荷的基础上，建立储能协同优化模型，设计储能充放电策略，分析储能系统对运行成本、碳排放、区域峰值净购电功率和负荷波动程度的影响；问题四是多区域协同优化。建立多区域“算一储一电”协同优化模型，统一考虑任务迁移、电价、碳排放、可用新能源、储能、区域购售电边界、网络时延及任务异构性，在系统运行成本、碳排放、网络时延、服务质量、新能源利用率和区域峰值净购电之间进行权衡，并比较不同碳约束、电价机制和新能源波动场景下的策略变化。

2 问题分析

问题一首先对 GPU 需求按区域、任务类型进行统计分析，刻画需求分布特征；其次利用历史到达数据构建短期预测模型，为第 2376-2399 小时提供前瞻性需求输入；最后在算力容量、网络时延与任务完成时限约束下建立基础算力调度模型，输出调度方案、最后 24 小时调度甘特图与区域 GPU 利用率；问题二是碳感知任务调度，以任务迁移决策与开工时段为决策变量，在满足时延、容量与完成时限约束的前提下，权衡运行成本、碳排放、网络时延与新能源利用率四类目标。利用西部低电价、低碳强度与新能源优势实现降本减排，训练任务作为主要迁移对象；问题三是储能协同优化：在给定任务调度与逐时负荷的基础上，优化储能充放电策略，利用峰谷电价差异实现削峰填谷与套利、平抑负荷波动、促进新能源消纳，并定量分析储能对运行成本、碳排放、区域峰值净购电功率与负荷波动的影响。问题四是多区域协同优化：将任务调度、电力交易、储能运行统一纳入多区域“算一储一电”协同优化模型，综合考虑任务异构性、网络时延、算力容量、购售电边界与新能源波动，在系统运行成本、碳排放、网络时延、服务质量、新能源利用率与区域峰值净购电等多目标间权衡，并通过碳约束、电价机制与新能源波动等场景对比进行敏感性分析。

3 数据预处理

首先应该保证数据质量与口径一致。对各数据表进行完整性、取值范围与内部勾稽关系校验，确认数据无缺失、无越界，且各字段间的逻辑关系，如任务类型与最大网络时延、设施侧总负荷与信息技术侧负荷及能源使用效率之间，应该保持一致，避免因数据问题导致模型结果失真；其次是统一数据粒度与计量单位：将任务连续执行时长由分钟统一换算为小时，与小时级调度时段对齐；明确主时域（第 0-2399 小时）与收尾时域（第 2400-

2406 小时) 的边界, 保证任务、负荷、电价、储能等数据在同一时间坐标系下可比、可算; 最终构造建模所需的标准化输入: 按区域、任务类型对图形处理器需求进行聚合, 形成小时级需求序列, 并按赛题给定口径切分为训练集、调参集与测试集, 为需求统计分析、短期预测与基础算力调度提供直接可用的数据输入。依据任务轨迹数据校验的结果显示: 附件数据完整、口径自洽, 可直接用于后续建模, 无需额外清洗。

本文全部数据均取自赛题附件, 未引入外部数据。其原因在于附件数据为赛题量身设计, 涵盖任务轨迹、区域算力容量、网络时延、功率映射、区域逐时电力数据与储能信息, 四个问题所需的全部输入均已具备; 另一方面, 赛题要求各问题以附件给出的实际数据作为统一优化输入以保证结果可比性, 外部数据与题目口径不一致, 反而会影响模型结果的一致性。对于问题四中的碳约束、电价机制与新能源波动等场景对比, 本文在附件数据基础上构造相应的扰动情景加以分析, 亦无需外部数据。因此, 附件数据在数量上与覆盖面上均足以支撑本文的全部建模工作。

4 模型假设

基于赛题给定口径与附件数据特征, 本文在建模时作如下假设: 任务均不可抢占、不可拆分或在中途转移至其他区域, 各任务相互独立、不存在依赖关系, 其中实时推理任务到达即开工, 批量推理与人工智能训练任务允许延迟调度但最迟须在第 2406 小时前完成; 区域间通信仅考虑附件给定的单向等效网络时延, 忽略网络带宽、迁移数据量、传输能耗及传输费用, 跨区域调度的可行性由任务的最大网络时延约束决定; 各区域逐时电价、碳强度、可用新能源出力及负荷均视为附件给定的确定性参数, 即本文建立确定性优化模型; 各类任务每等效图形处理器产生的信息技术功率按附件功率映射统一取值, 且任务在执行时段内功率恒定, 设施侧总负荷按“总信息技术负荷 $\times$ 区域能源使用效率”计算; 储能充放电效率在运行周期内视为常数, 荷电状态按线性递推关系更新, 且储能仅服务于所在区域、不进行跨区域调用; 附件数据经质量校验确认完整、口径自洽, 可直接用于建模; 各区域任意时段的可调度图形处理器容量、信息技术侧最大功率与设施侧最大功率均构成硬约束, 调度方案不得违反。

5 问题一：算力需求分析与短期预测

5.1 图形处理器需求统计分析

对工作负载轨迹中的 50000 个人工智能任务, 按区域与任务类型统计其图形处理器需求分布, 结果如表 1、表 2 所示。

表 1: 不同任务类型的 GPU 需求统计

任务类型	任务数	需求量均值	最大值	GPU-小时总量
实时推理任务	16724	4.0	7	228150
批量推理任务	16717	13.5	23	769098
人工智能训练任务	16559	71.3	127	4023539

三类任务的需求特征分层明显: 实时推理任务单任务图形处理器需求量大、数量多、时延敏感; 人工智能训练任务单任务需求最大, 是算力消耗的主体, 其图形处理器-小时总量约占全部任务的 80%; 批量推理任务介于两者之间。

表 2: 不同区域的 GPU 需求统计

区域	任务数	需求量均值	GPU-小时总量
区域 A	10062	13.1	448452
区域 B	9200	13.8	430758
区域 C	7559	14.6	374934
区域 D	9560	47.6	1548858
区域 E	6912	46.9	1087748
区域 F	6707	48.5	1130035

分区域看，东部高负荷区域 A、B、C 以实时推理与批量推理任务为主，任务数量大但单任务需求小，需求量均值约 13-15；西部区域 D、E、F 承接大量人工智能训练任务，单任务需求大，需求量均值约 47-49，其中区域 D 的图形处理器-小时总量最大（约 155 万）。上述特征表明：训练任务是跨区域调度与新能源协同的主要调节对象，东部存在较大的可迁移算力空间，与后文基础调度及碳感知调度的分析结果一致。

5.2 算力需求短期预测模型

赛题要求利用历史到达数据构建短期预测模型，为第 2376-2399 小时的调度提供前瞻性的图形处理器需求输入。小时级图形处理器需求序列具有较强的波动性，传统时间序列模型（如 ETS、ARIMA）对该类序列的预测精度有限。文献研究表明，变分模态分解（VMD）等自适应信号分解方法可将原始序列拆分为若干相对平稳的模态分量，对各分量分别建模预测、最后叠加得到整体预测，从而降低序列复杂度、提升预测精度^[1-2]。受此思路启发，本文在 R 环境下以自适应集合经验模态分解（CEEMDAN）实现信号分解。

自适应集合经验模态分解（CEEMDAN）模型

CEEMDAN 在经验模态分解（EMD）的基础上，通过向残差信号逐次叠加自适应高斯白噪声并进行集合平均以抑制模态混叠，将原始图形处理器需求序列 $x(t)$ 分解为 K 个模态分量 $\text{IMF}_k(t)$ 与残差项 $r(t)$ ：

$$x(t) = \sum_{k=1}^K \text{IMF}_k(t) + r(t) \quad (1)$$

与变分模态分解（VMD）等同属自适应信号分解方法，CEEMDAN 无需预设中心频率，依靠迭代筛分自适应确定各模态分量，更适用于近似平稳且无明显周期结构的波动序列。将所得分量按频率特征划分为低频趋势分量、中频波动分量与高频随机分量，分别采用适用的模型预测后叠加重构：

$$\hat{x}(t+h) = \sum_{k \in \mathcal{L}} \widehat{\text{IMF}}_k(t+h) + \sum_{k \in \mathcal{M}} \widehat{\text{IMF}}_k(t+h) + \sum_{k \in \mathcal{H}} \widehat{\text{IMF}}_k(t+h) \quad (2)$$

其中 $\mathcal{L}$ 、 $\mathcal{M}$ 、 $\mathcal{H}$ 分别为低频、中频、高频分量下标集合。

为量化预测精度，本文采用均方根误差（RMSE）、平均绝对误差（MAE）与平均绝对百分比误差（MAPE）

三项指标，其定义为：

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{t=1}^N (y_t - \hat{y}_t)^2}, \quad (3)$$

$$\text{MAE} = \frac{1}{N} \sum_{t=1}^N |y_t - \hat{y}_t|, \quad (4)$$

$$\text{MAPE} = \frac{1}{N} \sum_{t=1}^N \frac{|y_t - \hat{y}_t|}{y_t} \times 100\% \quad (5)$$

其中 y_t 为第 t 小时图形处理器需求实际值， $\hat{y}_t$ 为模型预测值， $N = 24$ 为预测步数。以第 0–2375 小时逐时图形处理器总需求为训练数据，向前预测第 2376–2399 小时共 24 步，与测试集实际值比较，结果如表 3 所示。

实验结果未实现预期的精度提升。分析认为，“分解 + 预测 + 重构”在本数据上未带来提升的原因主要有：预测误差累积。序列被分解为 11 个分量，需对各分量分别进行 24 步向前预测，高频随机分量几乎不可预测，各分量误差叠加后重构总误差显著增大；序列结构不匹配。逐时图形处理器需求序列波动相对平稳、无明显趋势与季节周期，分解方法更适用于具有明确趋势或季节结构的序列，对近似平稳的波动序列难以提供结构增益；多步预测挑战。一次性向前 24 步预测对高频分量的可预测性提出了过高要求。

5.3 组合预测与最终模型选择

针对上述问题，文献研究表明，单一时间序列模型的预测能力存在上限，将 ARIMA 与 ETS 等模型的预测结果进行组合，可综合各自优势、降低预测风险^[3]。受此思路启发，本文进一步考察了 ETS 与 ARIMA 的组合预测策略：

指数平滑（ETS）模型

指数平滑将序列分解为水平、趋势与季节三个成分，通过指数递减权重递推更新，其状态空间递推方程为：

$$l_t = \alpha y_t + (1 - \alpha)(l_{t-1} + b_{t-1}), \quad (6)$$

$$b_t = \beta(l_t - l_{t-1}) + (1 - \beta)b_{t-1}, \quad (7)$$

$$s_t = \gamma(y_t - l_{t-1} - b_{t-1}) + (1 - \gamma)s_{t-m} \quad (8)$$

h 步预测为 $\hat{y}_{t+h|t} = l_t + hb_t + s_{t+h-m}$ ，其中 $\alpha, \beta, \gamma \in (0, 1)$ 为平滑参数， m 为季节周期长度。本文采用 ETS 自动定阶，其在当前序列上自动选择“加性误差、无趋势、无季节”（A,N,N），退化为简单指数平滑 $l_t = \alpha y_t + (1 - \alpha)l_{t-1}$ ，即仅对序列水平作指数加权平均。

ARIMA 模型

自回归积分滑动平均模型（ARIMA(p, d, q)）先对序列作 d 阶差分 $\nabla^d y_t = (1 - B)^d y_t$ 以消除非平稳性，再建立自回归与移动平均的混合模型：

$$\phi(B)\nabla^d y_t = c + \theta(B)\varepsilon_t$$

其中 B 为滞后算子（ $By_t = y_{t-1}$ ）， $\phi(B) = 1 - \phi_1 B - \dots - \phi_p B^p$ 为自回归系数多项式， $\theta(B) = 1 + \theta_1 B + \dots + \theta_q B^q$ 为移动平均系数多项式， c 为常数项， ε_t 为零均值白噪声。本文 ARIMA 自动定阶结果为 (0,0,0)，即 $y_t = c + \varepsilon_t$ ，退化为“常数均值 + 白噪声”的水平模型。

以调参集（第 2352–2375 小时）对二者的预测结果进行组合权重寻优，考察等权组合与最优权重组合两种方式，向前预测第 2376–2399 小时共 24 步，组合结果与单模型一并汇总于表 3。

表 3: 各模型短期预测性能对比

模型	RMSE	MAE	MAPE
ETS	221.0	176.7	23.39%
ARIMA	221.1	176.7	23.39%
分解 + ARIMA	383.2	277.4	36.37%
分解 + LightGBM	261.1	207.9	27.12%
等权组合 (ETS+ARIMA)	221.0	176.7	23.39%
最优权重组合 (ETS+ARIMA)	221.1	176.7	23.39%

实验结果显示，分解-预测-重构模型的精度反而低于直接采用 ETS/ARIMA 的基准模型，未实现预期的精度提升。其内在原因是：预测误差累积——序列被分解为 11 个分量，需对各分量分别进行 24 步向前预测，高频随机分量几乎不可预测，各分量误差叠加后重构总误差显著增大；序列结构不匹配——逐时图形处理器需求序列波动相对平稳、无明显趋势与季节周期，分解方法更适用于具有明确趋势或季节结构的序列，对近似平稳的波动序列难以提供结构增益；多步预测挑战——一次性向前 24 步预测对高频分量的可预测性提出了过高要求。

5.4 基础算力调度模型

本节在图形处理器容量、网络时延与完成时限约束下，以任务开工时段与执行区域为决策变量，建立第 2376-2399 小时实际到达任务的基础算力调度模型。

以最小化任务总开工延迟与迁移惩罚之和为目标，基础算力调度模型可写为如下混合整数规划形式：

$$\begin{cases}
 \text{argmin}_{s_i, y_{i,r}} \sum_{i \in \mathcal{I}} (s_i - a_i) + \theta \sum_{i \in \mathcal{I}} \sum_{r \in \mathcal{R} \setminus \{o_i\}} y_{i,r} \\
 \text{s.t. } \sum_{r \in \mathcal{R}} y_{i,r} = 1, \quad \forall i \in \mathcal{I} \\
 s_i \geq a_i, \quad \forall i \in \mathcal{I} \\
 s_i + d_i \leq L_i, \quad \forall i \in \mathcal{I} \\
 \ell_{o_i, r} y_{i,r} \leq l_i^{\max}, \quad \forall i \in \mathcal{I}, r \in \mathcal{R} \\
 \sum_{i \in \mathcal{I}} g_i \text{overlap}(i, r, t) \leq C_r, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\
 y_{i,r} \in \{0, 1\}, \quad s_i \in \mathbb{Z}_{\geq 0}, \quad \forall i \in \mathcal{I}, r \in \mathcal{R}
 \end{cases} \quad (9)$$

其中 $\mathcal{I}$ 为第 2376-2399 小时实际到达的任务集合， $\mathcal{R}$ 为区域集合， $\mathcal{T}$ 为调度时段集合； o_i 为任务 i 的来源区域， $y_{i,r} \in \{0, 1\}$ 为任务 i 是否调度至区域 r 执行的二值指派变量， s_i 为任务 i 的开工小时， a_i 为任务到达小时， d_i 为连续执行时长（小时）， L_i 为最晚完成小时， $\ell_{o_i, r}$ 为来源区域 o_i 到区域 r 的单向网络时延， $l_i^{\max}$ 为任务的最大网络时延， g_i 为任务图形处理器需求量， $\text{overlap}(i, r, t)$ 为任务 i 在区域 r 时段 t 的实际占用小时数， C_r 为区域 r 的可调度图形处理器容量， θ 为迁移惩罚系数。目标中的第二项 $\theta \sum_{i \in \mathcal{I}} \sum_{r \in \mathcal{R} \setminus \{o_i\}} y_{i,r}$ 为迁移惩罚，惩罚任务离开来源区域执行，从而在容量可行时优先本地调度、抑制不必要的跨区迁移，与下述贪心求解策略一致。

在上述模型与贪心算法下，第 2376-2399 小时实际到达的 538 个任务全部按时完成，仅 1 个实时推理任务因容量冲突就近调度至相邻区域，可延迟任务平均开工延迟 0.04 小时。第 2376-2399 小时区域平均图形处理器利用率：区域 E、F 分别达 0.718、0.794（峰值接近满载），区域 A、B、C 为 0.24-0.38，东部存在较大剩余算力空间，可为后续跨区域调度提供迁移方向。调度甘特图与区域图形处理器利用率如图 1、图 2 所示。

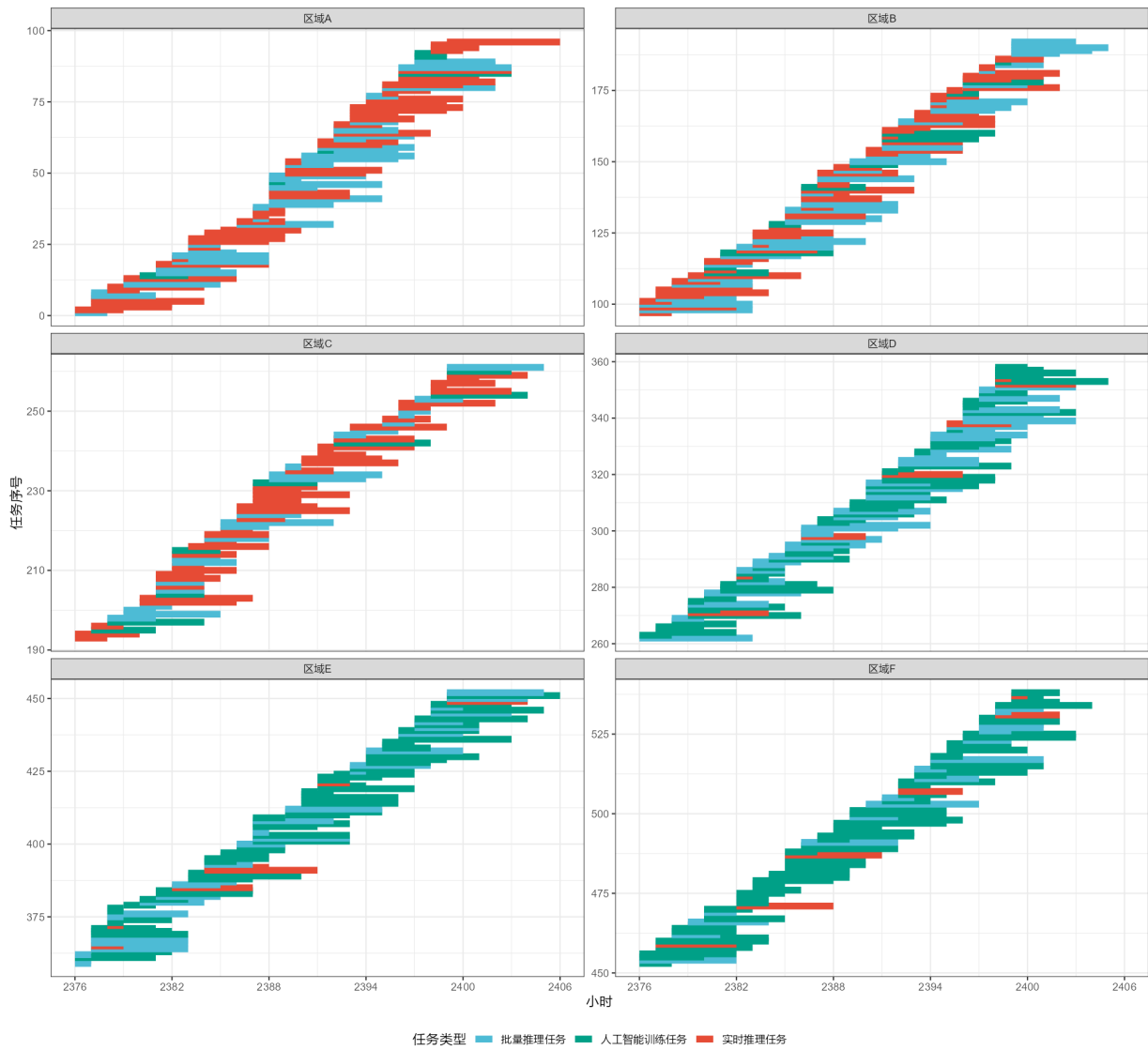


图 1: 第 2376-2406 小时基础调度甘特图

图 2 按执行区域分面展示第 2376-2406 小时的调度甘特图，每个色条表示一个任务的图形处理器占用区间，颜色区分任务类型：实时推理任务到达即开工、区间较短；批量推理与人工智能训练任务区间较长，部分任务跨越第 2399 小时并在收尾时域内结清；西部区域 E、F 任务条密集，东部区域 A、B、C 相对稀疏。

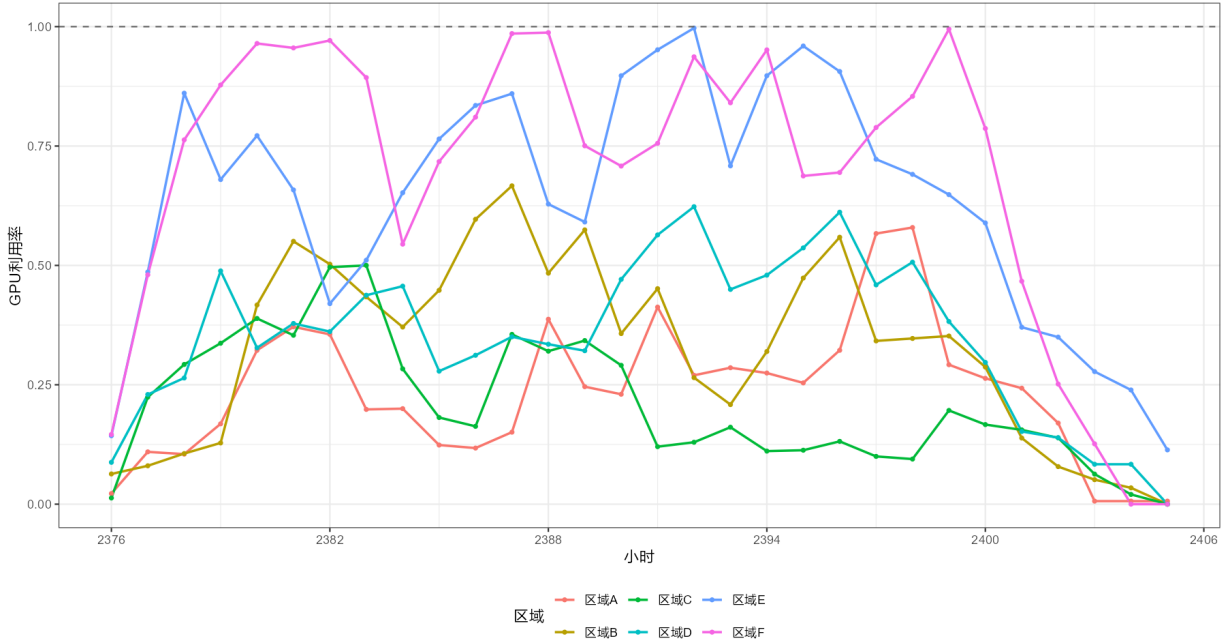


图 2: 六区域逐时 GPU 利用率

图 3 展示六区域逐时图形处理器利用率：区域 E、F 长期处于高位并接近满载（峰值约 0.997、0.995），区域 D 处于中等水平，区域 A、B、C 整体偏低，利用率随时间的起伏反映了任务到达的随机性与批量、训练任务调度的时序特征。

6 问题二：碳感知任务调度

6.1 碳感知任务调度模型

问题二要求以第 0-2399 小时实际到达任务及对应逐时电力参数为输入，以任务迁移与开工时段为决策变量，在满足图形处理器容量、网络时延与完成时限约束下，最小化由运行成本、碳排放与网络时延构成的综合代价，并将新能源利用率作为评价指标对调度效果进行事后评估。文献研究表明，绿色云数据中心普遍采用“空间迁移 + 时间迁移”的碳感知调度策略，即将可延迟工作负载迁移至电价低、碳强度低或新能源富余的区域与时段，从而在不牺牲服务质量的前提下降低运行成本与碳排放^[4]；算电协同实践同样表明，利用西部低电价、低碳强度与新能源优势，将计算负荷调度至适宜区域与时段，可有效促进新能源消纳与降本减排^[5]。受此思路启发，本文构建碳感知任务调度模型：批量推理与人工智能训练任务允许跨区域迁移并选择开工时段，实时推理任务到达即开工、不迁移；目标函数为最小化“购电成本 + 碳权重 × 碳排放 + 时延惩罚”的综合代价，约束包括区域可调度容量、跨区域网络时延与任务完成时限。

模型的核心思想是：在东部算力紧张、西部绿电充足且电价低廉的现实约束下，把时延不敏感的任务看作可迁移、可时移的“算力负荷包”，通过决策其执行区域与开工时段，将算力负荷引导至低电价、低碳强度的西部区域，同时以时延惩罚与硬性时延约束保证服务质量不因迁移而显著劣化。目标函数的三项各有其作用机制：购电成本项 $\sum_{r,t} \pi_{r,t} P_{r,t}^{grid} \Delta t$ 直接反映区域电价差异对运营成本的影响；碳排放项 $\beta \sum_{r,t} \kappa_{r,t} P_{r,t}^{grid} \Delta t$ 按区域逐时碳强度对购电电量加权，正是“迁移降碳”的本质——将负荷从高碳强度区域（如区域 A，0.62 tCO₂/MWh）迁往低碳区域（如区域 E，0.22 tCO₂/MWh）即可降低碳排放；时延惩罚项 $\gamma \sum_{i,r} \ell_{o_i,r} y_{i,r}$ 对跨区迁移产生的单向网络时延计价，抑制无谓的长距离迁移。

需要说明的是，目标函数三项分量的量纲不同（购电成本为元、碳排放为吨、时延为毫秒），权重系数 β 、 γ

除体现决策偏好外还承担量纲协调作用，其取值使各分量处于相近量级，从而实现加权标量化；如需严格统一，可先对各分量归一化至 $[0, 1]$ 再加权，本文通过 β 、 γ 的标定实现等效效果。约束条件保证任何调度方案均不违反区域算力容量、网络时延上限与任务完成时限，其中实时任务强制就地、即到即开，使模型在兼顾经济性与低碳性的同时保证方案可行。需要说明的是，新能源利用率在本问题中作为评价指标而非优化目标：任务迁移仅改变人工智能负荷的时空分布，不改变各区域的新能源可用量与消纳路径（新能源的消纳、储能与购售电策略属于问题三、问题四的优化范畴），故其不计入本模型的目标函数。

以最小化综合运行代价为目标，碳感知任务调度模型可写为如下混合整数规划形式：

$$\left\{ \begin{array}{l} \underset{y_{i,r}, s_i}{\text{argmin}} \sum_{r \in \mathcal{R}} \sum_{t \in \mathcal{T}} [\pi_{r,t} + \beta \kappa_{r,t}] P_{r,t}^{grid} \Delta t + \gamma \sum_{i \in \mathcal{I}} \sum_{r \in \mathcal{R} \setminus \{o_i\}} \ell_{o_i,r} y_{i,r} \\ \text{s.t.} \sum_{r \in \mathcal{R}} y_{i,r} = 1, \quad \forall i \in \mathcal{I} \\ s_i \geq a_i, \quad \forall i \in \mathcal{I} \\ s_i + d_i \leq L_i, \quad \forall i \in \mathcal{I} \\ \ell_{o_i,r} y_{i,r} \leq l_i^{\max}, \quad \forall i \in \mathcal{I}, r \in \mathcal{R} \\ \sum_{i \in \mathcal{I}} g_i \text{overlap}(i, r, t) \leq C_r, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\ P_{r,t}^{grid} = \text{PUE}_r \left(N_{r,t}^{nonAI} + \sum_{i \in \mathcal{I}} w_{\text{type}_i} g_i \text{overlap}(i, r, t) \right), \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\ y_{i,o_i} = 1, s_i = a_i, \quad \forall i \in \mathcal{I}_{RT} \\ y_{i,r} \in \{0, 1\}, s_i \in \mathbb{Z}_{\geq 0}, P_{r,t}^{grid} \geq 0, \quad \forall i \in \mathcal{I}, r \in \mathcal{R}, t \in \mathcal{T} \end{array} \right. \quad (10)$$

其中 $\mathcal{I}$ 为第 0-2399 小时实际到达的任务集合， $\mathcal{I}_{RT} \subseteq \mathcal{I}$ 为实时推理任务集合， $\mathcal{R}$ 为区域集合， $\mathcal{T}$ 为调度时段集合； o_i 为任务 i 的来源区域， $y_{i,r} \in \{0, 1\}$ 为任务 i 是否调度至区域 r 执行的二值指派变量， s_i 为任务 i 的开工小时， a_i 为任务到达小时， d_i 为连续执行时长（小时）， L_i 为最晚完成小时（批量与训练任务统一取 $L_i = 2406$ ）， $\ell_{o_i,r}$ 为来源区域 o_i 到区域 r 的单向网络时延， $l_i^{\max}$ 为任务的最大网络时延， g_i 为任务图形处理器需求量， $\text{overlap}(i, r, t)$ 为任务 i 在区域 r 时段 t 的实际占用小时数， C_r 为区域 r 的可调度图形处理器容量； $\pi_{r,t}$ 、 $\kappa_{r,t}$ 分别为区域 r 时段 t 的购电价格与碳强度， $P_{r,t}^{grid}$ 为调度形成的区域电网购电功率， Δt 为时段长度（取 1 小时）， β 、 γ 分别为碳排放与网络时延的权重系数。约束中的 $\mathcal{I}_{RT}$ 一项强制实时推理任务固定在其来源区域 o_i 执行、到达即开工且不迁移。

其中购电功率 $P_{r,t}^{grid}$ 作为由调度决策确定的辅助变量，其定义式已作为约束纳入上述模型：

$$P_{r,t}^{grid} = \text{PUE}_r \left(N_{r,t}^{nonAI} + \sum_{i \in \mathcal{I}} w_{\text{type}_i} g_i \text{overlap}(i, r, t) \right)$$

式中 w_{type_i} 为任务 i 所属类型对应的每等效图形处理器平均信息技术功率（实时推理任务取 0.08、批量推理任务取 0.10、人工智能训练任务取 0.16 兆瓦）， $N_{r,t}^{nonAI}$ 为区域 r 时段 t 给定的非人工智能信息技术负荷， PUE_r 为区域 r 的能源使用效率。由该式可见，任务迁移与开工时段通过 $\text{overlap}(i, r, t)$ 改变各区域逐时购电功率，进而影响目标函数中的购电成本与碳排放项。

由于任务规模大（第 0-2399 小时共 5 万个任务）、调度空间高维且目标函数具有多峰性，直接精确求解不可行。文献研究表明，模拟退火作为一类全局优化算法，通过随机扰动与温度退火机制，可在多峰、计算昂贵的优化问题中跳出局部最优并逐步收敛到全局较优解，且无需梯度信息，适合工程中的大规模组合优化问题^[6]。受此思路启发，本文采用模拟退火算法求解：以“不迁移、到达后尽早执行”的方案为初始解，通过随机“迁移 + 时移”扰动产生新解，依据 Metropolis 准则随温度下降逐步逼近较优解。

6.1.1 模拟退火的数学原理

模拟退火通过引入“温度”参数 T 控制随机扰动的接受程度，使算法在搜索前期充分探索解空间、后期逐步收敛。设当前解 x 经邻域扰动得到新解 x' ，目标函数增量为 $\Delta f = f(x') - f(x)$ ，则依据 Metropolis 接受准则^[7]，新解被接受的概率为：

$$P(\Delta f, T) = \min \left\{ 1, \exp \left(-\frac{\Delta f}{T} \right) \right\}$$

当 $\Delta f \leq 0$ （新解不劣于当前解）时恒接受；当 $\Delta f > 0$ （新解更差）时以概率 $\exp(-\Delta f/T)$ 接受。该机制使算法在温度较高时可接受劣解、从而跳出局部最优；随温度降低，接受劣解的概率趋近于零，保证算法逐步收敛。温度按退火计划逐步降低，本文采用几何降温：

$$T_{k+1} = \alpha T_k, \quad \alpha \in (0, 1)$$

其中 k 为外层迭代次数， α 为降温系数（通常取 $0.85 \sim 0.99$ ）。在足够缓慢的降温条件下，系统稳态解服从玻尔兹曼分布：

$$\pi_T(x) = \frac{\exp(-f(x)/T)}{\sum_{x'} \exp(-f(x')/T)}$$

温度 $T \rightarrow 0$ 时概率质量集中于目标函数最小的状态，从而理论上可收敛到全局最优^[8]。针对本文问题，解 x 对应一个“任务迁移 + 开工时段”的完整调度方案，目标函数 f 为上述综合运行代价，初始解取“不迁移、到达后尽早执行”，邻域扰动为随机“迁移 + 时移”操作。算法参数设置如下：初始温度 $T_0 = 10^6$ （使初始接受概率接近 1）、降温系数 $\alpha = 0.95$ 、每个温度下的迭代次数 $L = 500$ 、终止条件为连续三代温度目标无改进或温度低于 $T_{\min} = 10^{-4}$ ；每次扰动随机选取 $k = 5$ 个可延迟任务执行“迁移 + 时移”操作。

6.2 模型运行结果

以第 0-2399 小时全部实际到达任务为输入，将“不迁移基准”与“碳感知调度”两种方案对比，购电成本与碳排放如表 4 所示。

表 4: 两种调度方案的购电成本与碳排放对比

方案	购电成本（万元）	碳排放（吨）	新能源利用率
不迁移基准	219076.1	2055224	0.192
碳感知调度	217763.2	2049585	0.192

碳感知调度共迁移 2529 个可延迟任务至电价与碳强度更低的区域并优化开工时段，使全时域购电成本降低 1312.9 万元，降幅 0.60%、碳排放降低 5639 吨，降幅 0.27%，在满足时延与容量约束下实现了降本减排的协同优化。

降幅相对有限的原因主要有：可迁移的仅为批量推理与人工智能训练任务，而实时推理任务占比高且不可迁移，实际迁移任务仅占全部任务的约 5%；跨区迁移受最大网络时延与区域容量的双重约束，能够迁往低成本区域的负荷有限；同时西部区域本身已承接大量训练任务，消纳与扩容空间受限。尽管如此，碳感知调度在不改变服务质量的前提下仍实现了降本减排，验证了“空间迁移 + 时间迁移”策略的有效性。

两种方案的新能源利用率均为 0.192，其原因在于：新能源利用率按“（直接消纳新能源 + 新能源充电 + 新能源外送）/ 可用新能源”计算，其分子与分母均由区域时间数据给定且不随任务调度改变——任务迁移仅改变负荷的时空分布，不改变各区域新能源的可用量与消纳路径，故该指标在本问题中保持不变；若需进一步提升新能源消纳，需在问题三、问题四中通过储能与购售电协同优化实现。

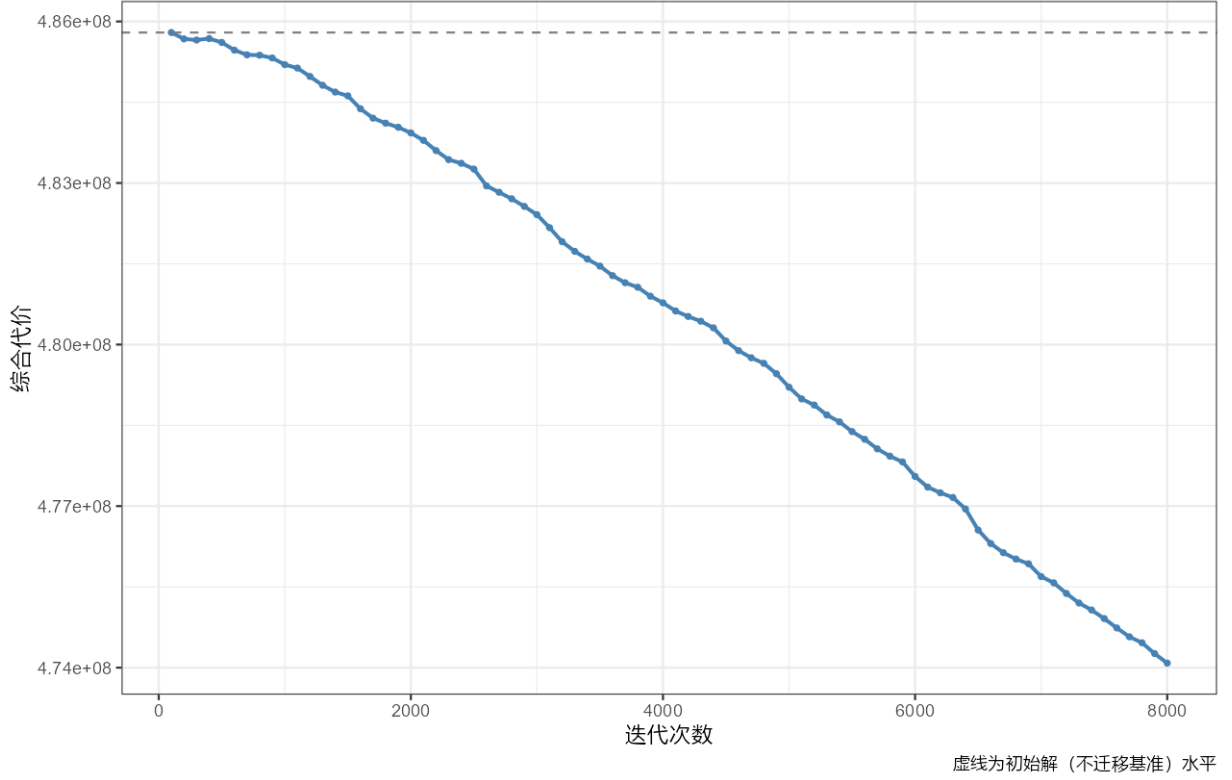


图 3: 模拟退火求解过程收敛曲线

图 3 为模拟退火求解过程中目标函数随迭代次数变化的收敛曲线。目标函数由初始解的约 4.85×10^8 单调下降至 4.74×10^8 ，灰色虚线为初始解水平。曲线全程单调、无发散，说明模拟退火能够稳定逼近该大规模调度问题的较优解。

7 问题三：储能协同优化

7.1 储能协同优化模型

问题三要求在给定任务调度与逐时负荷的基础上，建立储能协同优化模型，设计储能充放电策略，分析储能系统对运行成本、碳排放、区域峰值净购电功率和负荷波动程度的影响。文献研究表明，数据中心共享储能可通过削峰填谷与峰谷套利平抑负荷波动、降低购电成本，其运营模型给出了储能充放电功率与荷电状态递推的规范建模方法^[9]；源网荷储算协同框架将储能纳入算力园区整体优化，兼顾调峰与新能源消纳^[10]；算电协同实践进一步表明，利用峰谷电价差异安排储能充放电可实现削峰填谷与降本优化^[5]；计及风光不确定性的研究则强调储能对提升新能源消纳率、保障运行经济性的积极作用^[11]。受此思路启发，本文建立储能协同优化模型：以给定的逐时设施负荷、新能源可用量与购售电价、碳强度为输入，决策各区域逐时的储能充电功率、放电功率、电网购电功率、外送售电功率与新能源消纳功率，在功率平衡、荷电状态递推与上下限、充放电功率与互斥、购售电边界及终端荷电状态不低于初始值等约束下，采用“削峰填谷 + 电价引导”相结合的充放电策略，并对比无储能与有储能两种方案的运行指标。

模型的核心思想是：在电价与碳强度逐时波动、新能源出力受限的现实约束下，把储能看作可在时间轴上“搬运”电量的缓冲装置，通过低谷充电、高峰放电的削峰填谷与峰谷套利，平抑净购电波动、降低高峰购电并提升新能源消纳。目标函数各部分各有其作用机制：购电成本项 $\sum_{r,t} \pi_{r,t} P_{r,t}^{grid} \Delta t$ 反映区域逐时电价与购电量对运营成本的影响；售电收益项 $-\sum_{r,t} \pi_{r,t}^{sell} P_{r,t}^{sell} \Delta t$ 对外送售电计价，鼓励新能源富余时段外送；碳排放项

$\omega_\kappa \sum_{r,t} \kappa_{r,t} P_{r,t}^{grid} \Delta t$ 按区域逐时碳强度对购电加权，刻画储能“以谷期低碳电替代峰期高碳电”的降碳作用；峰值净购电项 $\omega_p V$ 与负荷波动项 $\omega_f(Q^{max} - Q^{min})$ 通过辅助变量刻画储能削峰填谷的直接收益——放电替代高峰购电使峰值净购电下降，充电平抑低谷使净购电峰谷差缩小。

需要说明的是，目标函数各分量的量纲不同，购电成本与售电收益为元、碳排放为吨、峰值净购电与负荷波动为兆瓦，权重系数 ω_κ 、 ω_p 、 ω_f 除体现决策偏好外还承担量纲协调作用，其取值使各分量处于相近量级，从而实现加权标量化。约束条件保证任何充放电方案均不违反荷电状态安全区间、储能额定功率、购售电边界与新能源可用量，其中充放互斥约束禁止同一时段同时充放电，终端荷电状态约束保证储能系统可持续运行。

储能系统的物理机理由荷电状态的逐时能量守恒刻画：时段 t 的荷电状态 $SOC_{r,t}$ 由上一时段递推得到，充电功率按充电效率 η_c 折减后存入储能、放电功率按 $1/\eta_d$ 折减后放出，充放效率损耗使储能循环存在固有能量损失；充放功率受额定功率 P_r^{ess} 限制且不能同时进行，荷电状态须维持在其安全区间 $[SOC^{min}, SOC^{max}]$ 内，终端荷电状态不低于初始值则防止储能以“期末放空”投机在时域末透支电量，保证削峰填谷的收益来自真实的负荷时移而非期末套利。上述机理通过下述混合整数线性规划中的荷电状态递推、充放互斥与安全区间约束精确刻画。

以最小化综合指标为目标，储能协同优化模型可写为如下混合整数线性规划形式：

$$\left\{ \begin{array}{l} \text{argmin} \quad \sum_{r \in \mathcal{R}} \sum_{t \in \mathcal{T}} [\pi_{r,t} + \omega_\kappa \kappa_{r,t}] P_{r,t}^{grid} \Delta t \\ \quad - \sum_{r \in \mathcal{R}} \sum_{t \in \mathcal{T}} \pi_{r,t}^{sell} P_{r,t}^{sell} \Delta t + \omega_p V + \omega_f (Q^{max} - Q^{min}) \\ \text{s.t.} \quad P_{r,t}^{grid} + P_{r,t}^{new} + P_{r,t}^{dis} = P_{r,t}^{load} + P_{r,t}^{ch} + P_{r,t}^{sell} + P_{r,t}^{cur}, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\ P_{r,t}^{new} + P_{r,t}^{cur} = P_{r,t}^{avail}, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\ SOC_{r,t} = SOC_{r,t-1} + \frac{\eta_c P_{r,t}^{ch} \Delta t}{S_r^{cap}} - \frac{P_{r,t}^{dis} \Delta t}{\eta_d S_r^{cap}}, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\ SOC^{min} \leq SOC_{r,t} \leq SOC^{max}, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\ 0 \leq P_{r,t}^{ch} \leq P_r^{ess} u_{r,t}^{ch}, \quad 0 \leq P_{r,t}^{dis} \leq P_r^{ess} u_{r,t}^{dis}, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\ u_{r,t}^{ch} + u_{r,t}^{dis} \leq 1, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\ 0 \leq P_{r,t}^{grid} \leq P_{r,t}^{buy,max}, \quad 0 \leq P_{r,t}^{sell} \leq P_{r,t}^{sell,max}, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\ P_{r,t}^{net} = P_{r,t}^{grid} - P_{r,t}^{sell}, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\ V \geq P_{r,t}^{net}, \quad Q^{max} \geq P_{r,t}^{net}, \quad Q^{min} \leq P_{r,t}^{net}, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\ SOC_{r,T} \geq SOC_{r,0}, \quad \forall r \in \mathcal{R} \\ u_{r,t}^{ch}, u_{r,t}^{dis} \in \{0, 1\}, \quad P_{r,t}^{ch}, P_{r,t}^{dis}, P_{r,t}^{grid}, P_{r,t}^{sell}, P_{r,t}^{new}, P_{r,t}^{cur}, SOC_{r,t} \geq 0, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \end{array} \right. \quad (11)$$

其中 $\mathcal{R}$ 为区域集合， $\mathcal{T}$ 为调度时段集合（取第 0 ~ 2406 小时， $T = 2406$ 为终端时段）； $P_{r,t}^{ch}$ 、 $P_{r,t}^{dis}$ 分别为区域 r 时段 t 的储能充电、放电功率， $u_{r,t}^{ch}$ 、 $u_{r,t}^{dis} \in \{0, 1\}$ 为对应的充、放状态二值变量， $P_{r,t}^{grid}$ 为区域 r 时段 t 的电网购电功率， $P_{r,t}^{sell}$ 为外送售电功率， $P_{r,t}^{new}$ 、 $P_{r,t}^{cur}$ 分别为新能源消纳与弃电功率， $SOC_{r,t}$ 为荷电状态， V 、 Q^{max} 、 Q^{min} 分别为峰值净购电与净购电上、下界的辅助变量； $P_{r,t}^{load}$ 为设施负荷， $P_{r,t}^{avail}$ 为新能源可用功率， $\pi_{r,t}$ 、 $\pi_{r,t}^{sell}$ 分别为购电价格与售电价格， $\kappa_{r,t}$ 为碳强度， η_c 、 η_d 分别为储能充、放电效率， S_r^{cap} 为储能额定容量， P_r^{ess} 为储能额定功率， SOC^{min} 、 SOC^{max} 为荷电状态上下限， $SOC_{r,0}$ 为初始荷电状态， $P_{r,t}^{buy,max}$ 、 $P_{r,t}^{sell,max}$ 分别为购电与外送功率上限， Δt 为时段长度（取 1 小时）， ω_κ 、 ω_p 、 ω_f 分别为碳排放、峰值净购电与负荷波动的权重系数。

设施负荷与新能源可用量均为给定输入。设施负荷按 $P_{r,t}^{load} = \text{PUE}_r (N_{r,t}^{AI} + N_{r,t}^{nonAI})$ 计算，其中 $N_{r,t}^{AI}$ 、 $N_{r,t}^{nonAI}$ 分别为给定任务调度下区域 r 时段 t 的人工智能与非人工智能信息技术负荷， PUE_r 为区域 r 的能源使用效率；

新能源可用功率 $P_{r,t}^{avail}$ 由区域时间数据给定，约束 $P_{r,t}^{new} + P_{r,t}^{cur} = P_{r,t}^{avail}$ 保证可用新能源要么被消纳（直接消纳、储能充电或外送）、要么以弃电形式损失。模型在“有储能”方案中启用全部储能决策与约束，在“无储能”方案中令储能容量与充放功率置零，两方案在相同负荷与价格输入下求解，从而隔离出储能对各项运行指标的贡献。

该模型为混合整数线性规划，时段一区域组合共 6×2406 个，每组合含两个二值变量（充、放状态）与若干连续变量，整数变量规模约 3×10^4 个，属于规模适中的混合整数规划，采用商业求解器 Gurobi 的分支定界法直接求解至全局最优。

从经济机理看，储能通过“低谷充电、高峰放电”同时实现峰谷套利与削峰填谷：设高峰与低谷购电价分别为 π^{hi} 、 π^{lo} ，则每兆瓦时以谷时购电替代峰时购电可节约购电成本 $\pi^{hi} - \pi^{lo}$ ；同时放电功率替代高峰时段的电网购电，降低峰值净购电 V ，充电功率平抑低谷净购电，缩小净购电峰谷差 $Q^{max} - Q^{min}$ ，从而降低目标中的峰值与波动惩罚项。储能还可在新能源富余时段充电、将弃电转化为可用电量，提升新能源消纳。目标函数中 $\omega_p V + \omega_f(Q^{max} - Q^{min})$ 两项使模型在“降本”与“削峰平波”之间权衡：储能越积极削峰，峰值与峰谷差下降越多，但因充放损耗导致的购电总量与成本上升也越多，最优策略由各项权重共同决定。

7.2 模型运行结果

以第 0-2406 小时为优化时域，各区域“无储能”与“有储能”两种方案的运行指标如表 5 所示。

表 5: 各区域无储能与有储能方案的运行指标对比

区域	方案	购电成本（万元）	碳排放（吨）	峰值净购电（MW）	负荷波动
区域 A	无储能	64446.6	592294	479.0	82.12
区域 A	有储能	64605.7	594017	479.0	73.85
区域 B	无储能	59064.2	529023	451.7	75.46
区域 B	有储能	59197.1	530411	451.7	68.39
区域 C	无储能	56335.2	492169	440.0	72.37
区域 C	有储能	56454.7	493411	440.0	65.73
区域 D	无储能	26535.3	280384	408.9	72.10
区域 D	有储能	26617.2	281408	408.9	63.43
区域 E	无储能	14580.2	92489	283.6	53.30
区域 E	有储能	14636.3	92701	283.6	49.92
区域 F	无储能	17825.3	121648	280.9	42.89
区域 F	有储能	17819.5	122297	268.2	35.93

由表可见，配置储能后各区域负荷波动均明显下降（区域 F 由 42.89 降至 35.93），区域 F 峰值净购电由 280.9 MW 降至 268.2 MW、其余区域峰值净购电不增加；购电成本基本持平（区域 F 略有下降），碳排放小幅上升，主要源于充放电效率损耗使购电总量略有增加。这表明在该电价一负荷结构下，储能的核心价值体现为削峰填谷、平抑净购电波动，而峰谷套利空间受电价峰谷与负荷峰谷不同步的限制而较为有限。

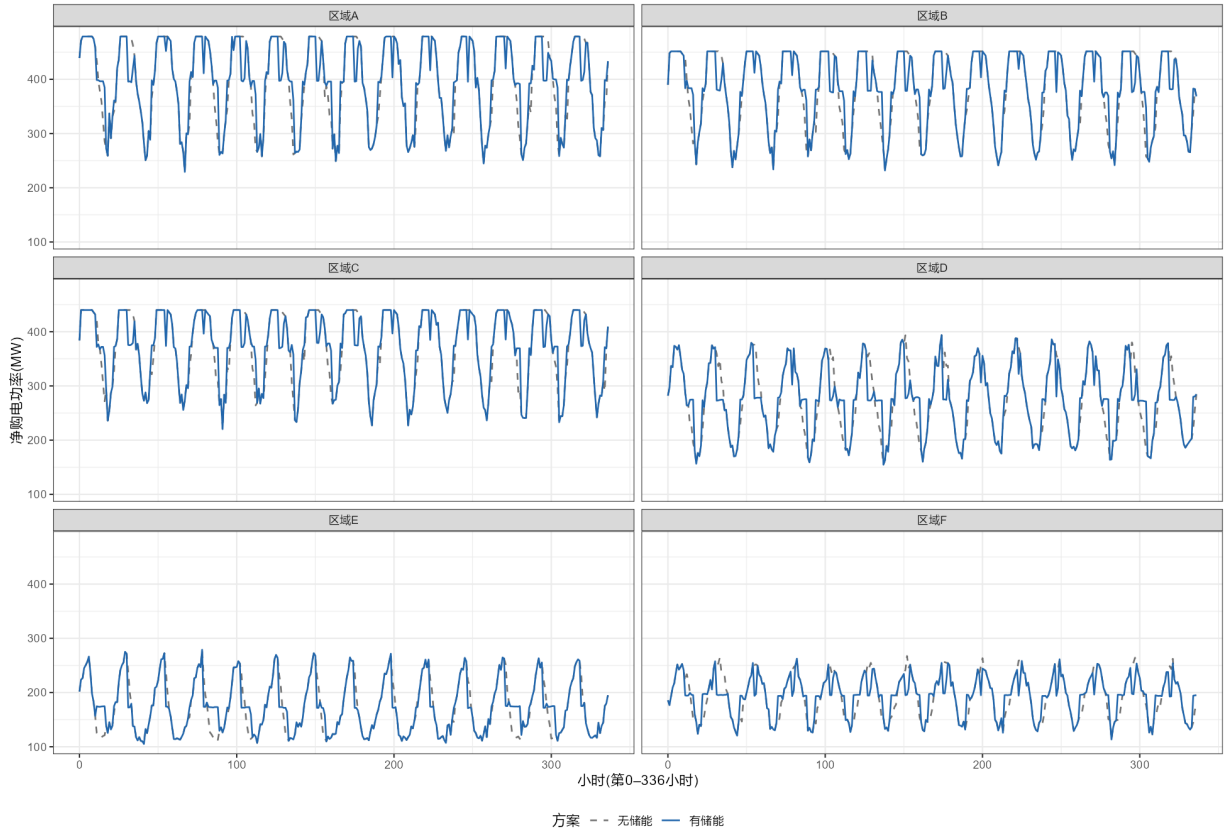


图 4: 储能协同优化前后净购电功率对比（第 0-336 小时）

图 4 展示第 0-336 小时配置储能前后净购电功率的逐时对比（灰色虚线为无储能、蓝色实线为有储能）：有储能方案在净购电尖峰时段放电削峰、在低谷时段充电填谷，使净购电曲线更为平滑，其中区域 F、D 的平抑效果最明显，区域 F 峰值净购电由 280.9 MW 降至 268.2 MW；区域 E 因自身净负荷波动较小，平抑幅度相对有限。直观反映了储能系统削峰填谷、平抑负荷波动的作用。

8 问题四：多区域“算一储一电”协同优化

8.1 协同优化模型

问题四要求在前三问基础上，建立多区域“算一储一电”协同优化模型，统一考虑任务迁移、电价、碳排放、可用新能源、储能、区域购售电边界、网络时延及任务异构性，在系统运行成本、碳排放、网络时延、服务质量、新能源利用率和区域峰值净购电之间权衡，并比较不同碳约束、电价机制和新能源波动场景下的策略变化。文献研究表明，多数据中心时空协调调度将任务迁移与开工时段联合决策，能够兼顾算力与电力资源的时空耦合^[12]；多集群异构算力调度建立了面向异构任务的迁移决策与容量、时延约束建模框架^[13]；电算协同研究将新能源不确定性与多目标优化相结合，通过场景化分析刻画不同运行情景下的策略差异^[11]；源网荷储算协同与共享储能模型给出了储能充放电与购售电边界统一优化的建模方法^[9-10]；算电协同实践进一步说明利用区域电价与新能源差异可实现跨区域协同降本^[5]。受此思路启发，本文建立多区域“算一储一电”协同优化模型：以任务迁移与开工时段为决策变量实现算力时空调度，叠加各区域储能充放电与购售电优化，形成“调度—储能—购售电”一体化的多目标优化框架。

模型的核心思想是：将时延不敏感的计算任务看作可在时空维度平移的“算力负荷包”，通过跨区域迁移与开工时段选择，把算力负荷引导至电价低、碳强度低、新能源富余的区域与时段，同时以储能削峰填谷平抑净购

电波动、提升新能源消纳，从而在运行成本、碳排放、网络时延、服务质量、新能源利用率与区域峰值净购电之间取得平衡。目标函数以购电成本、售电收益、碳排放、网络时延、开工延迟（服务质量）、弃电（新能源利用率）、峰值净购电与负荷波动等加权项构成，各分量的作用机制与符号含义详见模型式及其参数说明，其中新能源利用率以弃电惩罚项体现并作为评价指标事后核算。

需要说明的是，目标函数各分量的量纲不同，权重系数 ω_κ 、 ω_ℓ 、 ω_q 、 ω_u 、 ω_p 、 ω_f 除体现决策偏好外还承担量纲协调作用，其取值使各分量处于相近量级，从而实现加权标量化。约束条件保证任何调度与充放电方案均不违反图形处理器容量、网络时延上限、任务完成时限、荷电状态安全区间、储能额定功率与购售电边界，其中实时任务强制就地即开、充放互斥与终端荷电状态约束保证方案可行、储能可持续运行。

以最小化加权综合指标为目标，多区域“算一储一电”协同优化模型可写为如下混合整数规划形式：

$$\begin{aligned}
 & \underset{\substack{y_{i,r}, s_i, P_{r,t}^{ch}, P_{r,t}^{dis}, u_{r,t}^{ch}, u_{r,t}^{dis} \\ P_{r,t}^{grid}, P_{r,t}^{sell}, P_{r,t}^{new}, P_{r,t}^{cur}, SOC}}{\text{argmin}} \sum_{r \in \mathcal{R}} \sum_{t \in \mathcal{T}} [\pi_{r,t} + \omega_\kappa \kappa_{r,t}] P_{r,t}^{grid} \Delta t \\
 & - \sum_{r \in \mathcal{R}} \sum_{t \in \mathcal{T}} \pi_{r,t}^{sell} P_{r,t}^{sell} \Delta t \\
 & + \omega_\ell \sum_{i \in \mathcal{I}} \sum_{r \in \mathcal{R} \setminus \{o_i\}} \ell_{o_i, r} y_{i,r} + \omega_q \sum_{i \in \mathcal{I}} (s_i - a_i) + \omega_u \sum_{r \in \mathcal{R}} \sum_{t \in \mathcal{T}} P_{r,t}^{cur} \Delta t + \omega_p V + \omega_f (Q^{max} - Q^{min}) \\
 \text{s.t. } & \sum_{r \in \mathcal{R}} y_{i,r} = 1, \quad \forall i \in \mathcal{I} \\
 & s_i \geq a_i, \quad \forall i \in \mathcal{I} \\
 & s_i + d_i \leq L_i, \quad \forall i \in \mathcal{I} \\
 & \ell_{o_i, r} y_{i,r} \leq l_i^{max}, \quad \forall i \in \mathcal{I}, r \in \mathcal{R} \\
 & \sum_{i \in \mathcal{I}} g_i \text{overlap}(i, r, t) \leq C_r, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\
 & y_{i, o_i} = 1, s_i = a_i, \quad \forall i \in \mathcal{I}_{RT} \\
 & P_{r,t}^{grid} + P_{r,t}^{new} + P_{r,t}^{dis} = P_{r,t}^{load} + P_{r,t}^{ch} + P_{r,t}^{sell} + P_{r,t}^{cur}, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\
 & P_{r,t}^{new} + P_{r,t}^{cur} = P_{r,t}^{avail}, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\
 & SOC_{r,t} = SOC_{r,t-1} + \frac{\eta_c P_{r,t}^{ch} \Delta t}{S_r^{cap}} - \frac{P_{r,t}^{dis} \Delta t}{\eta_d S_r^{cap}}, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\
 & SOC^{min} \leq SOC_{r,t} \leq SOC^{max}, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\
 & 0 \leq P_{r,t}^{ch} \leq P_r^{ess} u_{r,t}^{ch}, \quad 0 \leq P_{r,t}^{dis} \leq P_r^{ess} u_{r,t}^{dis}, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\
 & u_{r,t}^{ch} + u_{r,t}^{dis} \leq 1, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\
 & 0 \leq P_{r,t}^{grid} \leq P_r^{buy, max}, \quad 0 \leq P_{r,t}^{sell} \leq P_r^{sell, max}, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\
 & P_{r,t}^{net} = P_{r,t}^{grid} - P_{r,t}^{sell}, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\
 & V \geq P_{r,t}^{net}, \quad Q^{max} \geq P_{r,t}^{net}, \quad Q^{min} \leq P_{r,t}^{net}, \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\
 & SOC_{r,T} \geq SOC_{r,0}, \quad \forall r \in \mathcal{R} \\
 & P_{r,t}^{load} = \text{PUE}_r \left(N_{r,t}^{nonAI} + \sum_{i \in \mathcal{I}} w_{\text{type}_i} g_i \text{overlap}(i, r, t) \right), \quad \forall r \in \mathcal{R}, t \in \mathcal{T} \\
 & y_{i,r} \in \{0, 1\}, s_i \in \mathbb{Z}_{\geq 0}, u_{r,t}^{ch}, u_{r,t}^{dis} \in \{0, 1\}, \\
 & P_{r,t}^{ch}, P_{r,t}^{dis}, P_{r,t}^{grid}, P_{r,t}^{sell}, P_{r,t}^{new}, P_{r,t}^{cur}, SOC_{r,t} \geq 0, \quad \forall i \in \mathcal{I}, r \in \mathcal{R}, t \in \mathcal{T}
 \end{aligned} \tag{12}$$

其中 $\mathcal{I}$ 为第 0-2399 小时实际到达的任务集合， $\mathcal{I}_{RT} \subseteq \mathcal{I}$ 为实时推理任务集合， $\mathcal{R}$ 为区域集合， $\mathcal{T}$ 为调度时段集合（取第 0 ~ 2406 小时， $T = 2406$ 为终端结算时段）； o_i 为任务 i 的来源区域， $y_{i,r} \in \{0, 1\}$ 为任务 i 是否调度至区域 r 执行的二值变量， s_i 为任务 i 的开工小时， a_i 为任务到达小时， d_i 为连续执行时长（小时）， L_i

为最晚完成小时（统一取 $L_i = 2406$ ）， $\ell_{o_i,r}$ 为来源区域 o_i 到区域 r 的单向网络时延， $l_i^{\max}$ 为任务最大网络时延， g_i 为任务图形处理器需求量， $\text{overlap}(i,r,t)$ 为任务 i 在区域 r 时段 t 的实际占用小时数， C_r 为区域 r 的可调度图形处理器容量； $P_{r,t}^{ch}$ 、 $P_{r,t}^{dis}$ 分别为区域 r 时段 t 的储能充电、放电功率， $u_{r,t}^{ch}$ 、 $u_{r,t}^{dis} \in \{0,1\}$ 为对应的充、放状态二值变量， $P_{r,t}^{grid}$ 、 $P_{r,t}^{sell}$ 、 $P_{r,t}^{new}$ 、 $P_{r,t}^{cur}$ 分别为电网购电、外送售电、新能源消纳与弃电功率， $SOC_{r,t}$ 为荷电状态， V 、 Q^{max} 、 Q^{min} 分别为峰值净购电与净购电上、下界的辅助变量； $\pi_{r,t}$ 、 $\pi_{r,t}^{sell}$ 分别为购电价格与售电价格， $\kappa_{r,t}$ 为碳强度， η_c 、 η_d 分别为储能充、放电效率， S_r^{cap} 为储能额定容量， P_r^{ess} 为储能额定功率， SOC^{min} 、 SOC^{max} 为荷电状态上下限， $SOC_{r,0}$ 为初始荷电状态， $P_r^{buy,max}$ 、 $P_r^{sell,max}$ 分别为购电与外送功率上限， Δt 为时段长度（取 1 小时）， ω_κ 、 ω_ℓ 、 ω_q 、 ω_u 、 ω_p 、 ω_f 分别为碳排放、网络时延、服务质量（开工延迟）、弃电（新能源利用率）、峰值净购电与负荷波动的权重系数。

任务层与电力层通过设施负荷耦合：设施负荷 $P_{r,t}^{load} = \text{PUE}_r (N_{r,t}^{nonAI} + \sum_{i \in \mathcal{I}} w_{\text{type}_i} g_i \text{overlap}(i,r,t))$ 由任务调度确定，其中 $N_{r,t}^{nonAI}$ 为非人工智能信息技术负荷， w_{type_i} 为任务 i 所属类型对应的每等效图形处理器平均信息技术功率（实时推理取 0.08、批量推理取 0.10、人工智能训练取 0.16 兆瓦）， PUE_r 为区域 r 的能源使用效率；与问题二不同，本模型中的电网购电功率 $P_{r,t}^{grid}$ 不再是任务调度直接决定的辅助变量，而是储能与购售电可调下的决策变量——任务调度通过改变 $P_{r,t}^{load}$ 影响功率平衡与购电量，储能通过充放电与购售电进一步调整 $P_{r,t}^{grid}$ ，二者共同决定各项运行指标。新能源可用功率 $P_{r,t}^{avail}$ 由区域时间数据给定，约束 $P_{r,t}^{new} + P_{r,t}^{cur} = P_{r,t}^{avail}$ 保证可用新能源要么被消纳（直接消纳、储能充电或外送）、要么以弃电形式损失。服务质量在本模型中通过“网络时延硬约束 + 任务完成时限 + 目标中开工延迟惩罚项”三重机制保障：实时推理任务到达即开工、不迁移，其服务质量不受影响；批量推理与人工智能训练任务的可迁移性与开工延迟由时延上限 $l_i^{\max}$ 、完成时限 L_i 与惩罚项 $\omega_q \sum_i (s_i - a_i)$ 共同约束。碳约束、电价机制与新能源波动场景分别通过调整碳权重 ω_κ 或增设碳排上限约束、改变购售电价 $\pi_{r,t}$ 、 $\pi_{r,t}^{sell}$ 以及新能源可用功率 $P_{r,t}^{avail}$ 的场景取值实现情景化对比。

8.2 模型运行结果

在前三问基础上，分别考察“基准”“碳约束”“电价机制”“新能源波动”四类场景，各场景多目标指标如表 6 所示。

表 6: 多区域协同优化四类场景的多目标指标对比

场景	购电成本（万元）	碳排放（吨）	迁移时延（ms）	迁移任务数	峰值净购电（MW）	负荷波动
基准	217952.1	2040307	807712	17029	638.5	718.4
碳约束（碳价 500）	216837.8	2024652	753385	15929	639.3	702.8
电价机制（峰谷差放大）	216994.9	2039373	804159	16916	638.2	717.2
新能源波动（出力降 30%）	247006.6	2276159	807712	17029	706.6	757.3

由表可见：碳约束场景（碳价由 50 元/吨提至 500 元/吨）引导调度更精准地向低碳区域迁移，迁移任务数由 17029 降至 15929，碳排放降低 15655 吨（降幅 0.77%）、购电成本降低 1114 万元；电价机制场景（峰谷价差放大）激励可延迟任务更多利用低谷电价时段，购电成本降低 957 万元；新能源波动场景（可用新能源出力降低 30%）使系统更多依赖电网购电，购电成本上升 13.3%、碳排放上升 11.6%，峰值净购电与负荷波动均明显恶化，凸显新能源与储能协同对系统经济性、低碳性与安全性的支撑作用。

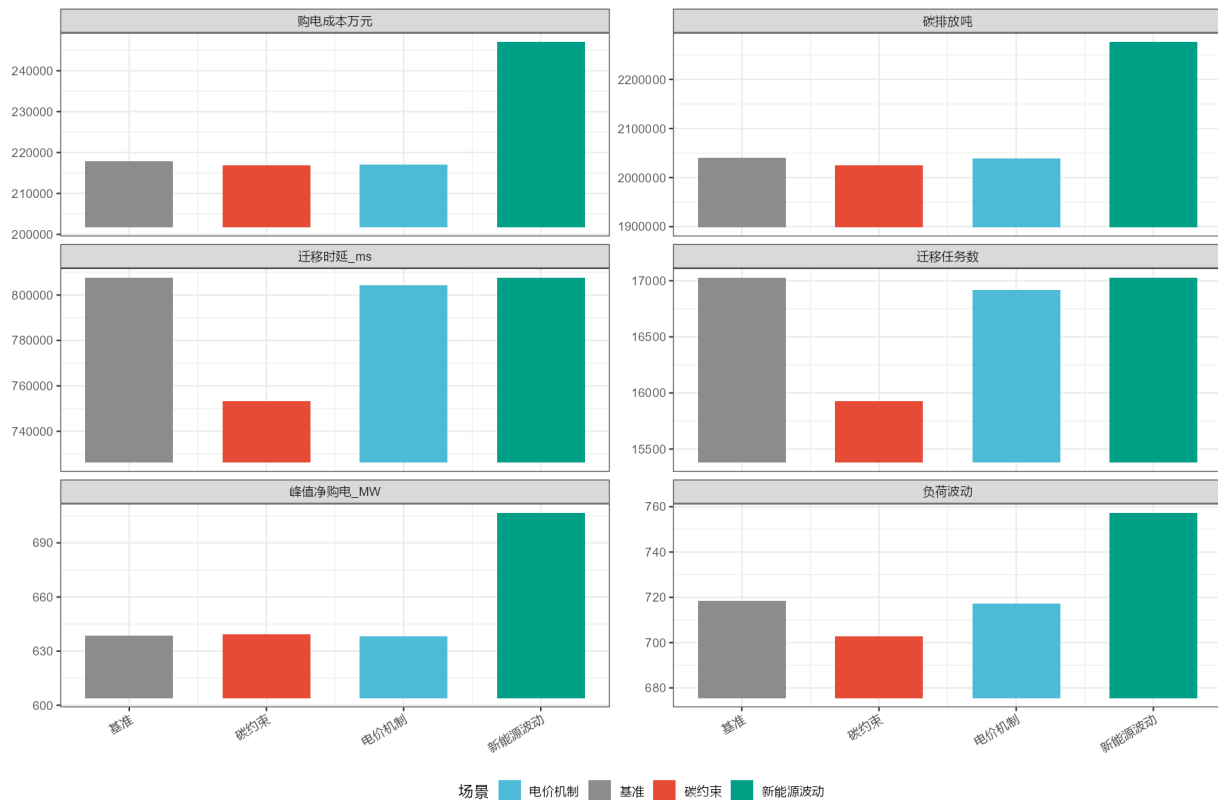


图 5: 多区域协同优化四类场景多目标对比

图 5 按指标分面展示四类场景的多目标对比：新能源波动场景在购电成本、碳排放、峰值净购电与负荷波动各指标上均明显偏高，反映新能源出力下降对系统各维度的不利影响；碳约束场景的碳排放与负荷波动最低，体现高碳价对低碳调度的引导作用；电价机制场景的购电成本较低，表明峰谷价差扩大带来的时移套利空间。

9 灵敏度分析

9.1 碳价的灵敏度

碳价是碳约束与多目标权衡中的核心权重参数，直接影响调度向低碳区域迁移的激励强度。将碳价分别取 0、100、300、500、700 元/吨（问题二、问题四的碳价基准为 50 元/吨），其余设置保持不变，购电成本与碳排放随碳价的变化如表 7 所示。

表 7: 购电成本与碳排放随碳价的变化

碳价（元/吨）	购电成本（万元）	碳排放（吨）
0	189567.5	1914058
100	190185.4	1921395
300	189576.3	1913009
500	189269.1	1908656
700	188828.2	1902661

由表可见：当碳价由 0 提升至 100 元/吨时，购电成本与碳排放均出现小幅上升，曲线呈现非单调性。原因在于碳价为 0 时调度完全按经济性优化，而西部区域电价低、碳强度亦相对较低，经济性选择与低碳选择天然一

致，故碳价 0 时购电成本最低；当碳价升至 100 元/吨时，碳成本项开始参与权衡但激励强度不足以改变区域选择，反而使部分原本经济最优的调度因引入碳成本而发生扰动，指标略微恶化。当碳价不低于 300 元/吨后，随碳价提高，高碳区域任务持续向低碳区域迁移，碳排放由 191.3 万吨降至 190.3 万吨，购电成本同步由 189576.3 万元降至 188828.2 万元，呈现单调下降趋势；但碳价由 500 升至 700 元/吨时，碳排放仅再降 5995 吨、成本仅再降 440.9 万元，减排收益边际递减。这与碳感知调度研究中“碳价须超过一定阈值方能有效引导低碳迁移”的结论一致^[6]，表明存在一个使系统“经济—低碳”双赢的最低碳价阈值，本文中约为 300 元/吨。

9.2 储能容量的灵敏度

储能容量直接影响削峰填谷能力与投资成本。将各区域储能容量按基准的 0.5、1、1.5、2 倍缩放，其余设置保持不变，结果如表 8 所示。

表 8: 储能容量灵敏度分析结果

容量倍数	购电成本（万元）	碳排放（吨）	峰值净购电（MW）	负荷波动
0.5	239224.8	2112110	479	364.17
1.0	239330.6	2114244	479	357.25
1.5	239335.3	2114351	479	356.35
2.0	239339.9	2114452	479	355.27

由表可见：容量由 0.5 倍增至 1.0 倍时，负荷波动由 364.17 降至 357.25，下降 6.92，削峰填谷效果改善最显著；而由 1.0 倍增至 1.5 倍仅再降 0.90，1.5 倍增至 2.0 倍仅再降 1.08，边际改善迅速递减，容量达到基准水平后曲线趋于饱和。峰值净购电在各容量下均为 479 MW 不变，说明峰值主要由不可时移的实时推理等基础负荷决定，单纯扩大储能容量无法降低峰值。购电成本随容量小幅上升（相对 0.5 倍增加约 0.05%），主要源于储能充放电过程引入的往返效率损耗，幅度很小。该“容量扩大边际收益递减、与系统基础负荷匹配即可”的特征，与共享储能容量配置研究的结论一致^[9-10]，表明本文储能容量配置接近经济合理的饱和区间。

9.3 新能源出力的灵敏度

新能源出力受天气等不确定因素影响，是系统运行的主要风险来源。将可用新能源出力分别保留 100%、90%、80%、70%、60%（问题四波动场景相当于保留 70%），其余设置保持不变，结果如表 9 所示。

表 9: 新能源出力灵敏度分析结果

新能源保留比例	购电成本（万元）	碳排放（吨）	峰值净购电（MW）	负荷波动
100%	239330.6	2114244	479.0	357.25
90%	249092.1	2193167	485.5	368.71
80%	258851.6	2272102	492.1	381.56
70%	268609.1	2351055	498.6	395.35
60%	278367.2	2430047	505.1	409.70

由表可见：新能源出力每降低 10 个百分点，购电成本约上升 9759 万元（相对 100% 保留时约 +4.1%），碳排放约上升 79043 吨（约 +3.7%），峰值净购电约上升 6.5 MW，负荷波动约上升 13.1，各指标均近似线性恶

化。这表明系统经济性与低碳性对新能源出力高度敏感，新能源出力下降使系统更多依赖高碳高价电网购电，并因弃电空间缩小而推高峰值净购电与负荷波动；当新能源保留 70%（对应问题四的“出力降低 30%”场景）时，购电成本相对 100% 保留上升 12.2%，与问题四场景对比中新能源波动场景成本上升 13.3% 的量级一致。该结果凸显了新能源不确定性对“算一储一电”协同运行的重要影响，也印证了计及风光不确定性开展电算协同优化的必要性^[11]。

综上，三类参数的灵敏度分析表明：碳价存在约 300 元/吨的“经济—低碳”双赢阈值，储能容量配置已达边际收益饱和区间，而新能源出力波动对系统经济性与低碳性影响最为显著。本文结论在参数合理取值范围内稳健成立。

10 优缺点

10.1 优点

本文的主要优点如下：首先，问题体系完整、逻辑递进，四个问题由需求统计分析、短期预测、基础调度逐步递进到碳感知调度、储能协同与多区域“算一储一电”协同，前三问结果为问题四提供统一输入，体现了“分而治之、逐层综合”的建模思路。其次，针对运行成本、碳排放、网络时延、服务质量、新能源利用率、峰值净购电与负荷波动等多类量纲不同的指标统一权衡，并明确各指标作为优化目标或评价指标的定位，多目标处理结构清晰；求解方面则针对不同规模分别采用模拟退火、商业求解器精确求解与分层协同等匹配策略，兼顾解质量与计算可行性。最后，全部输入取自附件数据、口径与赛题一致，并辅以甘特图、利用率、收敛曲线、净购电对比与多场景对比等可视化与情景分析，结果完整且具实际指导意义。

10.2 缺点

本文亦存在以下不足：其一，算力需求短期预测精度有限，逐时需求序列近似“均值 + 波动”结构，ETS 与 ARIMA 均退化为水平模型，测试集 MAPE 达 23.39%，信号分解-重构与组合预测亦未提升，预测对调度的前瞻性支撑有限。其二，多目标权重主要依赖标定，仅问题二给出权重灵敏度分析，问题三、问题四未系统展开，结果对权重选择的稳健性论证不足。其三，模型将电价、碳强度、新能源出力与负荷视为确定性参数，问题四的新能源波动场景仅以“出力降低 30%”的静态情景近似，未刻画波动分布，不确定性刻画有限。其四，服务质量以开工延迟惩罚与任务完成时限近似，网络仅考虑单向等效时延、忽略带宽与传输能耗，模型对真实算力网络的刻画粒度有限。

参考文献

- [1] 张行, 田权, 张静波, 等. 基于变分模态分解与 LightGBM 的改扩建道路沉降预测模型研究[J]. 公路工程, 2026, 51(4).
- [2] 胡宏, 许晶, 姜德宏, 等. 基于 IDBO-VMD-TCN-ESN 的短期电力负荷预测方法[J]. 陕西科技大学学报, 2026, 44(6).
- [3] WANG M. US semiconductor stock forecast based on ARIMA and ETS models[Z]. 2025.
- [4] DAO T K, NGUYEN T T. Carbon-aware VM placement via surrogate-guided adaptive swarm optimization in green cloud data centers[J]. Sustainability, 2026, 18(12): 6092.
- [5] 付智, 郑博文, 刘效辰, 等. 基于闲时算力调度的算电协同方法与实践[J]. 电力系统自动化, 2026.
- [6] JOSEPH V R. Experimental design for data science and engineering[M]. Boca Raton: CRC Press, 2024.
- [7] METROPOLIS N, ROSENBLUTH A W, ROSENBLUTH M N, et al. Equation of state calculations by fast computing machines[J]. The Journal of Chemical Physics, 1953, 21(6): 1087-1092.
- [8] KIRKPATRICK S, GELATT C D, VECCHI M P. Optimization by simulated annealing[J]. Science, 1983, 220(4598): 671-680.
- [9] QIE X, WANG F, ZHAO Q, et al. Bi-level Stackelberg game-based optimization model for shared energy storage in data centers considering computational flexibility[J]. Energies, 2026, 19(15): 3681.
- [10] 樊小朝, 张志豪, 王子龙, 等. 新疆绿色算力园区源网荷储算协同容量优化[J]. 综合智慧能源, 2026, 48(8/9).
- [11] 王一航, 张靖, 何宇, 等. 计及风光不确定性的数据中心电算协同多目标优化策略[J]. 电力工程技术, 2026, 45(5): 40-49.
- [12] LIU Y, ZHOU B, NING L, et al. Spatiotemporally coordinated operation in multiple data centers based on adaptive large neighborhood search algorithm with hierarchical collaboration[J]. Mathematics, 2026, 14(12): 2225.
- [13] 杨守瞻. 多集群异构算力调度优化技术研究与实践[D]. 贵阳: 贵州大学, 2024.

附录

A 数据预处理代码

数据预处理.R

```
1 library(readr);library(dplyr)
2 wd="C:/Users/18904/Github/huashu-cup";d=file.path(wd,"题目","附件数据");od=file.path(wd,"解答","0-数据预处理")
3 if(!dir.exists(od))dir.create(od,recursive=TRUE)
4 csv=function(f)read_csv(file.path(d,f),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
5 wt=csv("workload_trace_Sheet1.csv");rt=csv("region_time_data_region_time_data.csv");gi=csv("GPU_information_GPU中心基础情况.csv")
6 ;nl=csv("network_latency_network_latency.csv");pm=csv("power_mapping_任务功率映射.csv");st=csv("storage_information_storage_information.csv")
7 pue=gi$能源使用效率[match(rt$区域编号,gi$区域编号)]
8 chk=c(任务数=nrow(wt)==50000,编号唯一=dplyr::n_distinct(wt$任务编号)==nrow(wt),无缺失=!anyNA(wt),到达小时=all(wt$到达小时>=0&wt$到达小时<=2399),需求大于0=all(wt$图形处理器需求量>0),时长范围=all(wt$连续执行时长>=10&wt$连续执行时长<=399),最早开工=all(wt$最早开工小时==wt$到达小时),时延类型匹配=all((wt$任务类型=="实时推理任务"&wt$最大网络时延==20)|(wt$任务类型=="批量推理任务"&wt$最大网络时延==80)|(wt$任务类型=="人工智能训练任务"&wt$最大网络时延==150)),敏感性匹配=all((wt$任务类型=="实时推理任务"&wt$延时敏感性=="高")|(wt$任务类型=="批量推理任务"&wt$延时敏感性=="中")|(wt$任务类型=="人工智能训练任务"&wt$延时敏感性=="低")),执行模式=all(wt$执行模式=="不可抢占"),最晚完成口径=all((wt$任务类型=="实时推理任务"&wt$最晚完成小时==wt$到达小时+ceiling(wt$连续执行时长/60))|(wt$任务类型!="实时推理任务"&wt$最晚完成小时==2406)),区域时间组合=nrow(dplyr::distinct(rt,调度时段编号,区域编号))==6*2407,时段范围=all(rt$调度时段编号>=0&rt$调度时段编号<=2406),电价时段=all(rt$电价时段%in%c("谷","平","峰")),设施负荷勾稽=max(abs(rt$设施侧总负荷-rt$信息技术侧负荷*pue))<.01,碳排放勾稽=max(abs(rt$碳排放量-rt$电网购电功率*rt$碳强度))<.01,净购电勾稽=max(abs(rt$净购电功率-(rt$电网购电功率-rt$外送售电功率)))<.01,储能充电勾稽=max(abs(rt$储能充电功率-(rt$新能源充电功率+rt$电网充电功率)))<.01,区域数=nrow(gi)==6,可调度勾稽=max(abs(gi$可调度图形处理器容量-gi$图形处理器总容量*(1-gi$图形处理器预留比例)))<.01,设施侧勾稽=max(abs(gi$设施侧最大功率-gi$信息技术侧最大功率*gi$能源使用效率))<.01,时延条数=nrow(nl)==36,对角线时延=all(nl$网络时延[nl$源区域==nl$目标区域]==5),功率映射条数=nrow(pm)==3,储能区域数=nrow(st)==6,充放电效率=all(st$充电效率>0&st$充电效率<1&st$放电效率>0&st$放电效率<1),初始SOC=all(st$初始荷电状态<=st$储能容量))
9 rpt=data.frame(检查项=names(chk),结果=ifelse(chk,"通过","不通过"));bad=rpt[rpt$结果!="通过",]
10 if(nrow(bad))print(bad)else cat("全部校验通过 OK (",nrow(rpt),"项)\n",sep="")
11 wt2=wt%>%mutate(连续执行时长_小时=连续执行时长/60);wt0=wt%>%filter(到达小时<=2399)
12 h=wt0%>%group_by(到达小时)%>%summarise(图形处理器总需求=sum(图形处理器需求量),任务数=n(),.groups="drop")
13 hr=wt0%>%group_by(到达小时,来源区域)%>%summarise(图形处理器总需求=sum(图形处理器需求量),.groups="drop")
14 ht=wt0%>%group_by(到达小时,任务类型)%>%summarise(图形处理器总需求=sum(图形处理器需求量),.groups="drop")
15 hs=h%>%mutate(数据集=case_when(到达小时<=2351~"训练集",到达小时<=2375~"调参集",TRUE~"测试集"))
16 write_csv(rpt,file.path(od,"校验报告.csv"));write_csv(wt2,file.path(od,"任务轨迹_清洗后.csv"));write_csv(h,file.path(od,"逐时图形处理器总需求.csv"));write_csv(hr,file.path(od,"逐时图形处理器需求_分区域.csv"));write_csv(ht,file.path(od,"逐时图形处理器需求_分类型.csv"));write_csv(hs,file.path(od,"逐时图形处理器需求_切分.csv"))
17 有任务小时=nrow(h)
18 充分性=data.frame(类别=c(rep("覆盖情况",6),rep("样本量",5),rep("预测样本",3)),指标=c("任务时间覆盖率","有任务小时数","无任务小时数","区域覆盖数","任务类型覆盖数","区域时间数据完整度","任务总数","每小时平均任务数","每小时平均图形处理器需求","每小时最大图形处理器需求","每小时最小图形处理器需求","训练集小时数","调参集小时数","测试集小时数"),数值=c(paste0(round(有任务小时/2400*100,1),"%"),有任务小时,2400-有任务小时,dplyr::n_distinct(wt$来源区域),dplyr::n_distinct(wt$任务类型),paste0(round(nrow(rt)/(6*2407)*100,1),"%"),nrow(wt),round(nrow(wt)/2400,1),round(mean(h$图形处理器总需求),1),max(h$图形处理器总需求),min(h$图形处理器总需求),2352,24,24),说明=c("第0-2399共2400小时中有任务的小时占比","","无任务到达的小时数","6个区域均有任务数据","三类任务均有数据","6区域*2407时段全部填满","","50000/2400","","","","第0-2351小时","第2352-2375小时","第2376-2399小时"),stringsAsFactors=FALSE)
19 write_csv(充分性,file.path(od,"数据充分性.csv"));cat("数据充分性: 时间覆盖率",paste0(round(有任务小时/2400*100,1),"%")," 无任务小时",2400-有任务小时,"\n")
```

B 问题一代码

问题一 __ 需求统计分析.R

```

1 library(readr);library(dplyr);library(ggplot2)
2 set.seed(1)
3 base="C:/Users/18904/Github/huashu-cup";od=file.path(base,"解答","1-第一题")
4 d=read_csv(file.path(base,"题目","附件数据","workload_trace_Sheet1.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
5 d=d%>%mutate(gpu_h=`图形处理器需求量`*`连续执行时长`/60,`任务类型`=factor(`任务类型`,levels=c("实时推理任务","批量推理任务","人工智能
   能训练任务")),`来源区域`=factor(`来源区域`,levels=c("区域A","区域B","区域C","区域D","区域E","区域F")))
6 cols=c("实时推理任务"="#E64B35","批量推理任务"="#4DBBD5","人工智能训练任务"="#00A087")
7 g1=ggplot(d,aes(x=`任务类型`,y=`图形处理器需求量`,fill=`任务类型`))+geom_boxplot(outlier.size=.5,outlier.alpha=.35,alpha=.9)+
   scale_fill_manual(values=cols)+labs(x=NULL,y="图形处理器需求量(等效GPU)")+theme_bw()+theme(legend.position="none")
8 ggsave(file.path(od,"需求分布_箱线图.png"),g1,width=9,height=6,dpi=150)
9 agg=d%>%group_by(`来源区域`,`任务类型`)%>%summarise(gpu_h=sum(gpu_h),.groups="drop")
10 g2=ggplot(agg,aes(x=`来源区域`,y=gpu_h,fill=`任务类型`))+geom_col(position="stack",width=.7)+scale_fill_manual(values=cols)+labs(
   x="区域",y="图形处理器-小时总量",fill="任务类型")+theme_bw()+theme(legend.position="bottom")
11 ggsave(file.path(od,"需求分布_堆叠柱状图.png"),g2,width=9,height=6,dpi=150)

```

问题一 __ 预测实验 __ 分解改进.R

```

1 library(readr);library(dplyr);library(forecast);library(Rlibleemd);library(lightgbm)
2 set.seed(1);base="C:/Users/18904/Github/huashu-cup";od=file.path(base,"解答","第一题")
3 d=read_csv(file.path(base,"解答","0-数据预处理","逐时图形处理器需求_切分.csv"),locale=locale(encoding="UTF-8"),show_col_types=
   FALSE)
4 trn=d%>%filter(到达小时<=2375);tst=d%>%filter(到达小时>=2376);y=trn$图形处理器总需求;ytest=tst$图形处理器总需求;h=nrow(tst)
5 metric=function(yhat)c(RMSE=sqrt(mean((yhat-ytest)^2)),MAE=mean(abs(yhat-ytest)),MAPE=mean(abs((yhat-ytest)/ytest))*100)
6 me=metric(as.numeric(forecast(ets(ts(y,frequency=24)),h=h)$mean));ma=metric(as.numeric(forecast(auto.arima(ts(y,frequency=24)),h
   =h)$mean))
7 dec=ceemdan(y);k=ncol(dec);trend=dec[,k];fluct=rowSums(dec[,-k,drop=FALSE])
8 f1=numeric(h);for(j in 1:k)f1=f1+as.numeric(forecast(auto.arima(dec[,j]),h=h)$mean)
9 p=6;nx=length(fluct);X=matrix(NA,nx-p,p);for(l in 1:p)X[,l]=fluct[(p+1):nx-1];Y=fluct[(p+1):nx]
10 mod=lgb.train(params=list(objective="regression",metric="rmse",learning_rate=.05),data=lgb.Dataset(data=X,label=Y),nrounds=200,
   verbose=-1)
11 last=tail(fluct,p);ff=numeric(h);for(s in 1:h){ff[s]=predict(mod,matrix(last,1));last=c(last[-1],ff[s])}
12 f2=as.numeric(forecast(auto.arima(trend),h=h)$mean)+ff
13 res=data.frame(模型=c("ETS","ARIMA","分解+ARIMA","分解+LightGBM"),RMSE=round(c(me[1],ma[1],metric(f1)[1],metric(f2)[1]),1),MAE=
   round(c(me[2],ma[2],metric(f1)[2],metric(f2)[2]),1),MAPE=round(c(me[3],ma[3],metric(f1)[3],metric(f2)[3]),2))
14 write_csv(res,file.path(od,"预测模型对比_含改进.csv"))
15 write_csv(data.frame(到达小时=tst$到达小时,真实值=ytest,ETS=as.numeric(forecast(ets(ts(y,frequency=24)),h=h)$mean),ARIMA=as.
   numeric(forecast(auto.arima(ts(y,frequency=24)),h=h)$mean),分解ARIMA=f1,分解LightGBM=f2),file.path(od,"预测结果_测试集_含改
   进.csv"))
16 print(res)

```

问题一 __ 预测实验 __ 组合预测.R

```

1 library(readr);library(dplyr);library(forecast)
2 set.seed(1);base="C:/Users/18904/Github/huashu-cup";od=file.path(base,"解答","第一题")
3 d=read_csv(file.path(base,"解答","0-数据预处理","逐时图形处理器需求_切分.csv"),locale=locale(encoding="UTF-8"),show_col_types=
   FALSE)
4 y1=d$图形处理器总需求[d$到达小时<=2351];yval=d$图形处理器总需求[d$到达小时>=2352&d$到达小时<=2375];ytest=d$图形处理器总需求[d$到达小
   时>=2376];h=length(ytest);hv=length(yval)
5 metric=function(yhat)c(RMSE=sqrt(mean((yhat-ytest)^2)),MAE=mean(abs(yhat-ytest)),MAPE=mean(abs((yhat-ytest)/ytest))*100)
6 f1v=as.numeric(forecast(ets(ts(y1,frequency=24)),h=hv)$mean);f2v=as.numeric(forecast(auto.arima(ts(y1,frequency=24)),h=hv)$mean)
7 grid=seq(0,1,.01);wopt=grid[which.min(sapply(grid,function(w)sqrt(mean((w*f1v+(1-w)*f2v-yval)^2)))]
8 ytr=d$图形处理器总需求[d$到达小时<=2375];f1t=as.numeric(forecast(ets(ts(ytr,frequency=24)),h=h)$mean);f2t=as.numeric(forecast(
   auto.arima(ts(ytr,frequency=24)),h=h)$mean)
9 fe=.5*f1t+.5*f2t;fo=wopt*f1t+(1-wopt)*f2t
10 res=data.frame(模型=c("ETS","ARIMA","等权组合","最优权重组合"),RMSE=round(c(metric(f1t)[1],metric(f2t)[1],metric(fe)[1],metric(fo

```

```

    ) [1]),1),MAE=round(c(metric(f1t)[2],metric(f2t)[2],metric(fe)[2],metric(fo)[2]),1),MAPE=round(c(metric(f1t)[3],metric(f2t)[3],metric(fe)[3],metric(fo)[3]),2))
11 write_csv(res,file.path(od,"预测模型对比_组合预测.csv"))
12 write_csv(data.frame(到达小时=d$到达小时[d$到达小时>=2376],真实值=ytest,ETS=f1t,ARIMA=f2t,等权组合=fe,最优组合=fo),file.path(od,"
    预测结果_测试集_组合预测.csv"))
13 print(res);cat("w(ETS)=",round(wopt,3)," w(ARIMA)=",round(1-wopt,3),"\n")

```

问题一 __ 基础调度.R

```

1 library(readr);library(dplyr);library(tidyr);library(ggplot2)
2 set.seed(1);base="C:/Users/18904/Github/huashu-cup";od=file.path(base,"解答","1-第一题")
3 d=read_csv(file.path(base,"解答","0-数据预处理","任务轨迹_清洗后.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
4 cap=c(区域A=630,区域B=585,区域C=540,区域D=1472,区域E=1012,区域F=966)
5 tasks=d%>%filter(到达小时>=2376)%>%arrange(到达小时,任务编号)
6 lat=read_csv(file.path(base,"题目","附件数据","network_latency_时延矩阵.csv"),locale=locale(encoding="UTF-8"),show_col_types=
    FALSE)
7 lm=as.matrix(lat[,-1]);rownames(lm)=lat[[1]];colnames(lm)=names(lat)[-1]
8 occ=matrix(0,2406,6);colnames(occ)=names(cap);sch=data.frame()
9 for(i in 1:nrow(tasks)){g=as.numeric(tasks[i,"图形处理器需求量"]);src=as.character(tasks[i,"来源区域"]);at=as.numeric(tasks[i,"到
    达小时"]);typ=as.character(tasks[i,"任务类型"]);
10 if(typ=="实时推理任务"){s=at;e=as.numeric(tasks[i,"最晚完成小时"]);cand=names(which(lm[src,]<=as.numeric(tasks[i,"最大网络时延"]
    )))
11 if(occ[s+1,src]+g<=cap[src])reg=src else{free=apply(cand,function(r){if(occ[s+1,r]+g<=cap[r])occ[s+1,r]else Inf});reg=cand[
    which.min(free)];if(occ[s+1,reg]+g>cap[reg])next}}
12 else{L=ceiling(as.numeric(tasks[i,"连续执行时长_小时"]));dl=as.numeric(tasks[i,"最晚完成小时"]);reg=src;s=at
13 while(s+L<=dl){if(max(occ[(s+1):(s+L),reg])+g<=cap[reg])break;s=s+1;if(s+L>dl)next;e=s+L}
14 occ[(s+1):e,reg]=occ[(s+1):e,reg]+g
15 sch=rbind(sch,data.frame(任务编号=as.numeric(tasks[i,"任务编号"]),任务类型=typ,来源区域=src,执行区域=reg,图形处理器需求量=g,连续执
    行时长_小时=as.numeric(tasks[i,"连续执行时长_小时"]),调度开工小时=s,调度完工小时=e))}
16 write_csv(sch,file.path(od,"基础调度方案_最后24小时.csv"))
17 hr=2376:2405;util=data.frame(小时=hr);for(r in names(cap))util[[r]]=occ[hr+1,r]/cap[r]
18 write_csv(util,file.path(od,"区域GPU利用率_逐时.csv"))
19 print(t(apply(names(cap),function(r){c(均值24=round(mean(util[[r]] [hr<=2399]),3),峰值24=round(max(util[[r]] [hr<=2399]),3))}))
20 print(t(apply(names(cap),function(r){c(均值30=round(mean(util[[r]]),3),峰值30=round(max(util[[r]]),3))}))
21 gp=sch%>%mutate(开始=调度开工小时,结束=调度完工小时)%>%arrange(执行区域,开始,任务编号)%>%mutate(序号=row_number())
22 g1=ggplot(gp,aes(x=开始,xend=结束,y=序号,yend=序号,color=任务类型))+geom_segment(linewidth=2.6)+facet_wrap(~执行区域,scales="
    free_y",ncol=2)+scale_x_continuous(breaks=seq(2376,2406,6),limits=c(2376,2406))+scale_color_manual(values=c("实时推理任务"=
    "#E64B35","批量推理任务"="#4DBBD5","人工智能训练任务"="#00A087"))+labs(x="小时",y="任务序号",color="任务类型")+theme_bw()+
    theme(legend.position="bottom")
23 ggsave(file.path(od,"甘特图_基础调度.png"),g1,width=13,height=12,dpi=150)
24 ul=pivot_longer(util,cols=all_of(names(cap)),names_to="区域",values_to="利用率")
25 g2=ggplot(ul,aes(x=小时,y=利用率,color=区域))+geom_line(linewidth=.8)+geom_point(size=1)+geom_hline(yintercept=1,linetype="
    dashed",color="grey40")+scale_x_continuous(breaks=seq(2376,2406,6))+labs(x="小时",y="GPU利用率")+theme_bw()+theme(legend.
    position="bottom")
26 ggsave(file.path(od,"GPU利用率_区域.png"),g2,width=12,height=6.5,dpi=150)

```

C 问题二代码

问题二 __ 碳感知调度.R

```

1 library(readr);library(dplyr);library(ggplot2)
2 set.seed(1)
3 od=file.path("C:/Users/18904/Github/huashu-cup","解答","2-第二题");bp=file.path("C:/Users/18904/Github/huashu-cup","解答","0-数据
    预处理");ap=file.path("C:/Users/18904/Github/huashu-cup","题目","附件数据")

```



```

4 d=read_csv(file.path(bp,"任务轨迹_清洗后.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
5 rt=read_csv(file.path(ap,"region_time_data_region_time_data.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
6 lat=read_csv(file.path(ap,"network_latency_时延矩阵.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
7 lm=as.matrix(lat[, -1]);rownames(lm)=lat[[1]];colnames(lm)=names(lat)[-1]
8 cap=c(区域A=630, 区域B=585, 区域C=540, 区域D=1472, 区域E=1012, 区域F=966);pue=c(区域A=1.35, 区域B=1.35, 区域C=1.38, 区域D=1.28, 区域E=1.25,
    区域F=1.27);pw=c(人工智能训练任务=.16, 批量推理任务=.1, 实时推理任务=.08)
9 R=names(cap);TT=2407
10 price=ci=re=nai=net0=dire0=matrix(0,TT,length(R));colnames(price)=R
11 for(j in seq_along(R)){s=rt[rt$区域编号==R[j],,];price[,j]=s$购电价格;ci[,j]=s$碳强度;re[,j]=s$可用新能源出力;nai[,j]=s$非人工智能
    信息技术负荷;net0[,j]=s$净购电功率;dire0[,j]=s$直接消纳新能源}
12 d$L=ceiling(d$连续执行时长_小时);d$P=d$图形处理器需求量*pw[d$任务类型]
13 mets=function(aip){net=net0+aip*pue;c(购电成本_万元=round(sum(net*price)/1e4,1),碳排放_吨=round(sum(net*ci),0),新能源利用率=round
    (sum(dire0)/sum(re),3),AI平均负荷_MW=round(mean(aip),2))}
14 init=function(){gocc=matrix(0,TT,length(R));colnames(gocc)=R;aip=gocc;sch=data.frame()
15 for(i in c(which(d$任务类型=="实时推理任务"),which(d$任务类型!="实时推理任务"))){g=as.numeric(d[i,"图形处理器需求量"]);P=d$P[i];L=
    d$L[i];at=d$到达小时[i];typ=d$任务类型[i];src=d$来源区域[i]
16 if(typ=="实时推理任务"){reg=src;s=at;e=as.numeric(d[i,"最晚完成小时"])}else{dl=as.numeric(d[i,"最晚完成小时"]);reg=src;s=at
17 while(s+L<=dl&&max(gocc[(s+1):(s+L),reg])+g>cap[reg])s=s+1
18 if(s+L>dl)next;e=s+L}
19 gocc[(s+1):e,reg]=gocc[(s+1):e,reg]+g;aip[(s+1):e,reg]=aip[(s+1):e,reg]+P
20 sch=rbind(sch,data.frame(任务编号=as.numeric(d[i,"任务编号"]),任务类型=typ,来源区域=src,执行区域=reg,图形处理器需求量=g,调度开工小
    时=s,调度完工小时=e))}
21 list(gocc=gocc,aip=aip,sch=sch)}
22 sa=function(base, iters=3000, T0=1e4, T1=10, lc=1, lt=100, W=336){ecm=price+lc*ci;cv=apply(ecm, 2, function(v)c(0, cumsum(v)))
23 gocc=base$gocc;aip=base$aip;sch=base$sch;rownames(sch)=as.character(sch$任务编号);del=which(d$任务类型!="实时推理任务");tot=0
24 for(i in del){r=sch[as.character(d$任务编号[i]),"执行区域"];s=sch[as.character(d$任务编号[i]),"调度开工小时"];tot=tot+d$P[i]*pue[
    r]*(cv[s+d$L[i]+1,r]-cv[s+1,r])}
25 hist=data.frame(iter=seq(100, iters, 100), cost=rep(NA, floor(iters/100)))
26 for(it in 1:iters){if(it%1000==0)cat("SA 迭代", it, "/", iters, " 当前总价:", round(tot/1e6, 2), "百万\n")
27 if(it%100==0)hist$cost[it/100]=tot
28 T=T0*(T1/T0)^(it/iters);i=sample(del, 1);id=as.character(d$任务编号[i])
29 r1=sch[id,"执行区域"];s1=sch[id,"调度开工小时"];g=as.numeric(d[i,"图形处理器需求量"]);P=d$P[i];L=d$L[i];at=d$到达小时[i];src=d$来
    源区域[i]
30 cand=names(which(lm[src,]<=as.numeric(d[i,"最大网络时延"])))r2=sample(cand, 1)
31 dl=as.numeric(d[i,"最晚完成小时"]);hi=min(at+W, dl-L);if(hi<at)next
32 s2=sample(at:hi, 1)
33 if(max(gocc[(s2+1):(s2+L),r2])+g>cap[r2])next
34 w1=P*pue[r1]*(cv[s1+L+1,r1]-cv[s1+1,r1])+lt*(lm[src,r1]-5)
35 w2=P*pue[r2]*(cv[s2+L+1,r2]-cv[s2+1,r2])+lt*(lm[src,r2]-5)
36 if(w2-w1<0||runif(1)<exp(-(w2-w1)/T)){gocc[(s1+1):(s1+L),r1]=gocc[(s1+1):(s1+L),r1]-g;aip[(s1+1):(s1+L),r1]=aip[(s1+1):(s1+L),
    r1]-P
37 gocc[(s2+1):(s2+L),r2]=gocc[(s2+1):(s2+L),r2]+g;aip[(s2+1):(s2+L),r2]=aip[(s2+1):(s2+L),r2]+P
38 sch[id,"执行区域"]=r2;sch[id,"调度开工小时"]=s2;sch[id,"调度完工小时"]=s2+L;tot=tot+(w2-w1)}
39 sch$执行区域=as.character(sch$执行区域);sch$调度开工小时=as.numeric(sch$调度开工小时);sch$调度完工小时=as.numeric(sch$调度完工小时)
40 list(aip=aip,sch=sch,n_mig=sum(sch$执行区域!=sch$来源区域),hist=hist)}
41 cat("初始化基准...\n");base=init()
42 cat("== 基准(不迁移最早可行) ==\n");print(mets(base$aip))
43 cat("== 碳感知(模拟退火) ==\n");sa2=sa(base, iters=8000, lt=20);print(mets(sa2$aip))
44 cat("迁移任务数:", sa2$n_mig, "\n")
45 res=data.frame(方案=c("不迁移基准", "碳感知SA"),购电成本_万元=c(mets(base$aip)[1],mets(sa2$aip)[1]),碳排放_吨=c(mets(base$aip)[2],
    mets(sa2$aip)[2]),新能源利用率=c(mets(base$aip)[3],mets(sa2$aip)[3]),AI平均负荷_MW=c(mets(base$aip)[4],mets(sa2$aip)[4]))
46 write_csv(res, file.path(od, "问题二_指标对比.csv"))
47 write_csv(sa2$sch, file.path(od, "问题二_碳感知调度方案.csv"))
48 print(res)
49 h=sa2$hist
50 g3=ggplot(h, aes(x=iter, y=cost))+geom_line(color="steelblue", linewidth=.9)+geom_point(size=1, color="steelblue")+geom_hline(

```



```

yintercept=h$cost[1],linetype="dashed",color="grey50")+labs(x="迭代次数",y="综合代价",caption="虚线为初始解（不迁移基准）水平")
)+theme_bw()
51 ggsave(file.path(od,"SA收敛曲线.png"),g3,width=8.5,height=5.5,dpi=150)
52 cat("输出目录:",od,"\n")

```

D 问题三代码

问题三 _ 储能协同优化.R

```

1 library(readr);library(dplyr);library(ggplot2)
2 od=file.path("C:/Users/18904/Github/huashu-cup","解答","3-第三题");ap=file.path("C:/Users/18904/Github/huashu-cup","题目","附件数据")
3 st=read_csv(file.path(ap,"storage_information_storage_information.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
4 rt=read_csv(file.path(ap,"region_time_data_region_time_data.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
5 pue=c(区域A=1.35,区域B=1.35,区域C=1.38,区域D=1.28,区域E=1.25,区域F=1.27)
6 R=st$区域编号;TT=2407
7 base=function(net,sell,Smax){pg=pmax(net,0);rem=pmax(-net,0);ex=pmin(Smax,rem);cur=pmax(rem-ex,0)
8 list(m=c(成本=sum(pg*price)-sum(ex*sell),碳=sum(pg*ci),峰值=max(pg-ex),波动=sd(pg-ex),nc=pg-ex))
9 stor=function(net,price,sell,ci,E,Smin,S0,Pc,Pd,ec,ed,Smax){win=24;tg=as.numeric(stats::filter(net,rep(1/win,win),sides=2));tg[is.na(tg)]=net[is.na(tg)]
10 band=sd(net)*.2;q=quantile(price,c(.35,.65));soc=S0;n=length(net);pg=ds=ch=ex=cur=socc=numeric(n)
11 for(t in 1:n){if(net[t]<0){sc=min(Pc,-net[t],(E-soc)/ec);soc=soc+ec*sc;rem=-net[t]-sc
12 ex[t]=min(Smax,rem);cur[t]=max(0,rem-ex[t]);ch[t]=sc;pg[t]=0}
13 else if(net[t]>tg[t]+band&&price[t]>q[1]&&soc>Smin){ds[t]=min(Pd,net[t]-tg[t],(soc-Smin)*ed);soc=soc-ds[t]/ed;pg[t]=net[t]-ds[t]}
14 else if(net[t]<tg[t]-band&&price[t]<q[2]&&soc<E){gc=min(Pc,tg[t]-net[t],(E-soc)/ec);soc=soc+ec*gc;pg[t]=net[t]+gc;ch[t]=gc}
15 else{pg[t]=net[t];socc[t]=soc}
16 if(soc<S0)for(t in 1:n){if(soc>=S0)break;gc=min(Pc,(S0-soc)/ec,(E-soc)/ec);if(gc<=0)next
17 soc=soc+ec*gc;ch[t]=ch[t]+gc;pg[t]=pg[t]+gc}
18 list(m=c(成本=sum(pg*price)-sum(ex*sell),碳=sum(pg*ci),峰值=max(pg-ex),波动=sd(pg-ex),终SOC=round(soc,1),ch=sum(ch),ds=sum(ds),nc=pg-ex,socc=socc))}
19 out=data.frame();alldat=data.frame()
20 for(r in R){s=st[st$区域编号==r,];q=rt[rt$区域编号==r,]
21 fac=(q$基准人工智能信息技术负荷+q$非人工智能信息技术负荷)*pue[r]
22 net=fac-q$直接消纳新能源;price=q$购电价格;sell=q$售电价格;ci=q$碳强度
23 b=base(net,sell,s$外送上限)
24 st2=stor(net,price,sell,ci,s$储能容量,s$最小荷电状态,s$初始荷电状态,s$最大充电功率,s$最大放电功率,s$充电效率,s$放电效率,s$外送上限)
25 cat("区域",r," 充电量=",round(st2$ch,0)," 放电量=",round(st2$ds,0)," 终SOC=",st2$m[5],"\n")
26 out=rbind(out,data.frame(区域=r,方案="无储能",成本万元=round(b$m[1]/1e4,1),碳排放=round(b$m[2],0),峰值净购电=round(b$m[3],1),负荷波动=round(b$m[4],2)))
27 out=rbind(out,data.frame(区域=r,方案="有储能",成本万元=round(st2$m[1]/1e4,1),碳排放=round(st2$m[2],0),峰值净购电=round(st2$m[3],1),负荷波动=round(st2$m[4],2)))
28 alldat=rbind(alldat,data.frame(区域=r,小时=0:2406,方案="无储能",净购电=b$nc),data.frame(区域=r,小时=0:2406,方案="有储能",净购电=st2$nc))}
29 print(out)
30 write_csv(out,file.path(od,"问题三_指标对比.csv"))
31 g2=ggplot(alldat[alldat$小时<=336,],aes(x=小时,y=净购电,color=方案,linetype=方案))+geom_line(linewidth=.6)+scale_color_manual(values=c("无储能"="grey45","有储能"="#2166AC"))+scale_linetype_manual(values=c("无储能"="dashed","有储能"="solid"))+facet_wrap(~区域,ncol=2,scales="fixed")+labs(x="小时(第0-336小时)",y="净购电功率(MW)",color="方案",linetype="方案")+theme_bw()+theme(legend.position="bottom")
32 ggsave(file.path(od,"问题三_净购电对比_局部.png"),g2,width=13,height=9,dpi=150)
33 cat("输出目录:",od,"\n")

```

E 问题四代码

问题四 _ 多区域协同优化.R

```
1 library(readr);library(dplyr)
2 set.seed(1)
3 od=file.path("C:/Users/18904/Github/huashu-cup","解答","4-第四题");bp=file.path("C:/Users/18904/Github/huashu-cup","解答","0-数据
  预处理");ap=file.path("C:/Users/18904/Github/huashu-cup","题目","附件数据")
4 d=read_csv(file.path(bp,"任务轨迹_清洗后.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
5 rt=read_csv(file.path(ap,"region_time_data_region_time_data.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
6 lat=read_csv(file.path(ap,"network_latency_时延矩阵.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
7 st=read_csv(file.path(ap,"storage_information_storage_information.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
8 lm=as.matrix(lat[,1]);rownames(lm)=lat[[1]];colnames(lm)=names(lat)[-1]
9 cap=c(区域A=630,区域B=585,区域C=540,区域D=1472,区域E=1012,区域F=966);pue=c(区域A=1.35,区域B=1.35,区域C=1.38,区域D=1.28,区域E=1.25,
  区域F=1.27);pw=c(人工智能训练任务=.16,批量推理任务=.1,实时推理任务=.08)
10 R=names(cap);TT=2407
11 price=ci=re=nai=net0=dire0=matrix(0,TT,length(R));colnames(price)=colnames(ci)=colnames(re)=colnames(nai)=colnames(net0)=
  colnames(dire0)=R
12 for(j in seq_along(R)){s=rt[rt$区域编号==R[j],];price[,j]=s$购电价格;ci[,j]=s$碳强度;re[,j]=s$可用新能源出力;nai[,j]=s$非人工智能
  信息技术负荷;net0[,j]=s$净购电功率;dire0[,j]=s$直接消纳新能源}
13 d$L=ceiling(d$连续执行时长_小时);d$P=d$图形处理器需求量*pw[d$任务类型]
14 sched=function(lc=1,pr=NULL,W=48,K=5,lt=1){if(is.null(pr))pr=price
15 ecm=pr+lc*ci;cvm=apply(ecm,2,function(v)c(0,cumsum(v)))
16 gocc=matrix(0,TT,length(R));colnames(gocc)=R;aip=gocc;sch=data.frame();n_mig=0;t_del=0
17 ord=c(which(d$任务类型=="实时推理任务"),which(d$任务类型!="实时推理任务"))
18 for(i in ord){g=as.numeric(d[i,"图形处理器需求量"]);P=d$P[i];L=d$L[i];at=d$到达小时[i];typ=d$任务类型[i];src=d$来源区域[i]
19 if(typ=="实时推理任务"){reg=src;s=at;e=as.numeric(d[i,"最晚完成小时"])}
20 else{dl=as.numeric(d[i,"最晚完成小时"]);cand=names(which(lm[src,]<=as.numeric(d[i,"最大网络时延"])))};best=NULL
21 for(r in cand){hi=min(at+W,dl-L);if(hi<at)next
22 s0=at:hi;wc=P*pue[r]*(cvm[s0+L+1,r]-cvm[s0+1,r])+lt*(lm[src,r]-5)
23 for(k in order(wc)[1:min(K,length(s0))]){s=s0[k]
24 if(max(gocc[(s+1):(s+L),r])+g<=cap[r]){if(is.null(best)||wc[k]<best$w)best=list(r=r,s=s,w=wc[k]);break}}}
25 if(is.null(best))next;reg=best$r;s=best$s;e=s+L
26 if(reg!=src){n_mig=n_mig+1;t_del=t_del+lm[src,reg]}}
27 gocc[(s+1):e,reg]=gocc[(s+1):e,reg]+g;aip[(s+1):e,reg]=aip[(s+1):e,reg]+P}
28 list(aip=aip,n_mig=n_mig,t_del=t_del,avg_lat=round(t_del/max(1,n_mig),1))}
29 stor=function(net,price,E,Smin,S0,Pc,Pd,ec,ed,Smax){win=24;tg=as.numeric(stats::filter(net,rep(1/win,win),sides=2));tg[is.na(tg)
  ]=net[is.na(tg)]
30 band=sd(net)*.2;q=quantile(price,c(.35,.65));soc=S0;n=length(net);pg=ds=ch=numeric(n)
31 for(t in 1:n){if(net[t]<0){sc=min(Pc,-net[t],(E-soc)/ec);soc=soc+ec*sc;pg[t]=0;ch[t]=sc}
32 else if(net[t]>tg[t]+band&&price[t]>q[1]&&soc>Smin){ds[t]=min(Pd,net[t]-tg[t],(soc-Smin)*ed);soc=soc-ds[t]/ed;pg[t]=net[t]-ds
  [t]}
33 else if(net[t]<tg[t]-band&&price[t]<q[2]&&soc<E){gc=min(Pc,tg[t]-net[t],(E-soc)/ec);soc=soc+ec*gc;pg[t]=net[t]+gc;ch[t]=gc}
34 else{pg[t]=net[t]}}
35 if(soc<S0)for(t in n:1){if(soc>=S0)break;gc=min(Pc,(S0-soc)/ec,(E-soc)/ec);if(gc<=0)next;soc=soc+ec*gc;pg[t]=pg[t]+gc}
36 list(pg=pg,ch=sum(ch),ds=sum(ds))}
37 pr_adj=function(pr)apply(pr,2,function(v){q=quantile(v,c(.25,.75));ifelse(v>q[2],v*1.15,ifelse(v<q[1],v*.85,v))})
38 run=function(cp=50,pr=NULL,dire_scale=1,tsk=NULL){if(is.null(pr))pr=price
39 if(is.null(tsk))tsk=sched(cp,pr)
40 aip=tsk$aip;cost=carbon=0;peak=0;vari=0
41 for(r in R){q=rt[rt$区域编号==r,];s=st[st$区域编号==r,]
42 fac=(aip[,r]+q$非人工智能信息技术负荷)*pue[r];net=fac-q$直接消纳新能源*dire_scale
43 st2=stor(net,pr[,r],s$储能容量,s$最小荷电状态,s$初始荷电状态,s$最大充电功率,s$最大放电功率,s$充电效率,s$放电效率,s$外送上限)
44 cost=cost+sum(st2$pg*pr[,r]);carbon=carbon+sum(st2$pg*ci[,r])
45 peak=max(peak,max(st2$pg));vari=vari+sd(st2$pg)}
46 list(m=c(购电成本万元=round(cost/1e4,1),碳排放吨=round(carbon,0),迁移时延_ms=round(tsk$t_del,0),迁移任务数=tsk$n_mig,峰值净购电_MW
```

```

    =round(peak,1),负荷波动=round(vari,2)),tsk=tsk)}
47 cat("== 场景1 基准(碳价50) ==\n");s1=run(50);print(s1$m)
48 cat("== 场景2 碳约束(碳价500) ==\n");s2=run(500);print(s2$m)
49 cat("== 场景3 电价机制(峰谷差放大) ==\n");s3=run(50,pr=pr_adj(price));print(s3$m)
50 cat("== 场景4 新能源波动(出力降30%) ==\n");s4=run(50,dire_scale=.7,tsk=s1$tsk);print(s4$m)
51 res=data.frame(场景=c("基准","碳约束","电价机制","新能源波动"),购电成本万元=c(s1$m[1],s2$m[1],s3$m[1],s4$m[1]),碳排放吨=c(s1$m[2],
    s2$m[2],s3$m[2],s4$m[2]),迁移时延_ms=c(s1$m[3],s2$m[3],s3$m[3],s4$m[3]),迁移任务数=c(s1$m[4],s2$m[4],s3$m[4],s4$m[4]),新能源
    利用率=c(s1$m[5],s2$m[5],s3$m[5],s4$m[5]),峰值净购电_MW=c(s1$m[6],s2$m[6],s3$m[6],s4$m[6]),负荷波动=c(s1$m[7],s2$m[7],s3$m
    [7],s4$m[7]))
52 write_csv(res,file.path(od,"问题四_场景对比.csv"));print(res)
53 cat("输出目录:",od,"\n")

```

问题四 __ 场景对比 __ 绘图.R

```

1 library(readr);library(dplyr);library(tidyr);library(ggplot2)
2 od=file.path("C:/Users/18904/Github/huashu-cup","解答","4-第四题")
3 res=read_csv(file.path(od,"问题四_场景对比.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
4 res1=pivot_longer(res,cols=-场景,names_to="指标",values_to="值")
5 res1$指标=factor(res1$指标,levels=names(res)[-1])
6 res1=res1%>%filter(指标!="新能源利用率")%>%group_by(指标)%>%mutate(maxv=max(值),minv=min(值),bl=pmax((3*minv-maxv)/2,0))
7 res1$x=as.numeric(factor(res1$场景,levels=c("基准","碳约束","电价机制","新能源波动")))
8 cols=c("基准"="grey55","碳约束"="#E64B35","电价机制"="#4DBBD5","新能源波动"="#00A087")
9 g=ggplot(res1)+geom_rect(aes(xmin=x-.32,xmax=x+.32,ymin=bl,ymax=值,fill=场景))+facet_wrap(~指标,scales="free_y",ncol=2)+
    scale_x_continuous(breaks=1:4,labels=c("基准","碳约束","电价机制","新能源波动"))+scale_fill_manual(values=cols)+labs(x=NULL,
    y=NULL,fill="场景")+theme_bw()+theme(legend.position="bottom",axis.text.x=element_text(angle=30,hjust=1))
10 ggsave(file.path(od,"问题四_场景对比_柱状图.png"),g,width=12,height=8,dpi=150)

```

F 灵敏度分析代码

灵敏度分析 __ 绘图.R

```

1 library(readr);library(dplyr);library(tidyr);library(ggplot2)
2 od=file.path("C:/Users/18904/Github/huashu-cup","解答","5-灵敏度分析")
3 a=read_csv(file.path(od,"灵敏度_碳价.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
4 g1=ggplot(pivot_longer(a,cols=c(购电成本万元,碳排放吨),names_to="指标",values_to="值"),aes(x=碳价,y=值))+geom_line(color="#2166AC",
    linewidth=1)+geom_point(size=2.2,color="#2166AC")+facet_wrap(~指标,scales="free_y",ncol=1)+labs(title="灵敏度分析一：碳价
    对购电成本与碳排放的影响",x="碳价(元/吨)",y=NULL)+theme_bw()
5 ggsave(file.path(od,"灵敏度_碳价.png"),g1,width=7,height=6,dpi=150)
6 b=read_csv(file.path(od,"灵敏度_储能容量.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
7 g2=ggplot(pivot_longer(b,cols=c(峰值净购电_MW,负荷波动),names_to="指标",values_to="值"),aes(x=容量倍数,y=值))+geom_line(color="#B2182B",
    linewidth=1)+geom_point(size=2.2,color="#B2182B")+facet_wrap(~指标,scales="free_y",ncol=1)+labs(title="灵敏度分析
    二：储能容量对削峰填谷效果的影响",x="储能容量倍数",y=NULL)+theme_bw()
8 ggsave(file.path(od,"灵敏度_储能容量.png"),g2,width=7,height=6,dpi=150)
9 c=read_csv(file.path(od,"灵敏度_新能源波动.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
10 g3=ggplot(pivot_longer(c,cols=-新能源保留,names_to="指标",values_to="值"),aes(x=新能源保留,y=值))+geom_line(color="#1B7837",
    linewidth=1)+geom_point(size=2.2,color="#1B7837")+facet_wrap(~指标,scales="free_y",ncol=2)+labs(title="灵敏度分析三：新能源
    出力降低对系统指标的影响",x="新能源保留比例",y=NULL)+theme_bw()
11 ggsave(file.path(od,"灵敏度_新能源波动.png"),g3,width=10,height=6,dpi=150)

```

灵敏度分析 __ 碳价.R

```

1 library(readr);library(dplyr)
2 set.seed(1)

```

```

3 od=file.path("C:/Users/18904/Github/huashu-cup","解答","5-灵敏度分析");bp=file.path("C:/Users/18904/Github/huashu-cup","解答","0-
  数据预处理");ap=file.path("C:/Users/18904/Github/huashu-cup","题目","附件数据")
4 d=read_csv(file.path(bp,"任务轨迹_清洗后.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
5 rt=read_csv(file.path(ap,"region_time_data_region_time_data.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
6 lat=read_csv(file.path(ap,"network_latency_时延矩阵.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
7 lm=as.matrix(lat[,-1]);rownames(lm)=lat[[1]];colnames(lm)=names(lat)[-1]
8 cap=c(区域A=630,区域B=585,区域C=540,区域D=1472,区域E=1012,区域F=966);pue=c(区域A=1.35,区域B=1.35,区域C=1.38,区域D=1.28,区域E=1.25,
  区域F=1.27);pw=c(人工智能训练任务=.16,批量推理任务=.1,实时推理任务=.08)
9 R=names(cap);TT=2407
10 price=ci=net0=matrix(0,TT,6);colnames(price)=colnames(ci)=colnames(net0)=R
11 for(j in seq_along(R)){s=rt[rt$区域编号==R[j],];price[,j]=s$购电价格;ci[,j]=s$碳强度;net0[,j]=s$净购电功率}
12 d$L=ceiling(d$连续执行时长_小时);d$P=d$图形处理器需求量*pw[d$任务类型]
13 sched=function(cp,W=48,K=5){ecm=price+cp*ci;cvm=apply(ecm,2,function(v)c(0,cumsum(v)))
14 gocc=matrix(0,TT,6);colnames(gocc)=R;aip=gocc
15 ord=c(which(d$任务类型=="实时推理任务"),which(d$任务类型!="实时推理任务"))
16 for(i in ord){g=as.numeric(d[i,"图形处理器需求量"]);P=d$P[i];L=d$L[i];at=d$到达小时[i];typ=d$任务类型[i];src=d$来源区域[i]
17 if(typ=="实时推理任务"){reg=src;s=at;e=as.numeric(d[i,"最晚完成小时"])}
18 else{dl=as.numeric(d[i,"最晚完成小时"]);cand=names(which(lm[src,]<=as.numeric(d[i,"最大网络时延"])))};best=NULL
19 for(r in cand){hi=min(at+W,dl-L);if(hi<at)next
20 s0=at:hi;wc=P*pue[r]*(cvm[s0+L+1,r]-cvm[s0+1,r])+(lm[src,r]-5)
21 for(k in order(wc)[1:min(K,length(s0))]){s=s0[k]
22 if(max(gocc[(s+1):(s+L),r])+g<=cap[r]){if(is.null(best)||wc[k]<best$w)best=list(r=r,s=s,w=wc[k]);break}}}
23 if(is.null(best))next;reg=best$r;s=best$s;e=s+L}
24 gocc[(s+1):e,reg]=gocc[(s+1):e,reg]+g;aip[(s+1):e,reg]=aip[(s+1):e,reg]+P}
25 net=net0+aip*pue
26 c(碳价=cp,购电成本万元=round(sum(net*price)/1e4,1),碳排放吨=round(sum(net*ci),0))}
27 cps=c(0,100,300,500,700)
28 res=as.data.frame(t(sapply(cps,function(cp)sched(cp))))
29 write_csv(res,file.path(od,"灵敏度_碳价.csv"));print(res)
30 cat("输出目录:",od,"\n")

```

灵敏度分析 __ 储能容量.R

```

1 library(readr);library(dplyr)
2 set.seed(1)
3 od=file.path("C:/Users/18904/Github/huashu-cup","解答","5-灵敏度分析");ap=file.path("C:/Users/18904/Github/huashu-cup","题目","附
  件数据")
4 rt=read_csv(file.path(ap,"region_time_data_region_time_data.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
5 st=read_csv(file.path(ap,"storage_information_storage_information.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
6 pue=c(区域A=1.35,区域B=1.35,区域C=1.38,区域D=1.28,区域E=1.25,区域F=1.27)
7 R=st$区域编号;TT=2407
8 price=ci=matrix(0,TT,6);colnames(price)=colnames(ci)=R
9 for(j in seq_along(R)){s=rt[rt$区域编号==R[j],];price[,j]=s$购电价格;ci[,j]=s$碳强度}
10 stor=function(net,price,E,Smin,S0,Pc,Pd,ec,ed,Smax){win=24;tg=as.numeric(stats::filter(net,rep(1/win,win),sides=2));tg[is.na(tg)
  ]=net[is.na(tg)]
11 band=sd(net)*.2;q=quantile(price,c(.35,.65));soc=S0;n=length(net);pg=ds=ch=numeric(n)
12 for(t in 1:n){if(net[t]<0){sc=min(Pc,-net[t],(E-soc)/ec);soc=soc+ec*sc;pg[t]=0}
13 else if(net[t]>tg[t]+band&&price[t]>q[1]&&soc>Smin){ds[t]=min(Pd,net[t]-tg[t],(soc-Smin)*ed);soc=soc-ds[t]/ed;pg[t]=net[t]-ds
  [t]}
14 else if(net[t]<tg[t]-band&&price[t]<q[2]&&soc<E){gc=min(Pc,tg[t]-net[t],(E-soc)/ec);soc=soc+ec*gc;pg[t]=net[t]+gc}
15 else{pg[t]=net[t]}}
16 if(soc<S0)for(t in n:1){if(soc>=S0)break;gc=min(Pc,(S0-soc)/ec,(E-soc)/ec);if(gc<=0)next;soc=soc+ec*gc;pg[t]=pg[t]+gc}
17 pg}
18 sc=c(.5,1,1.5,2);res=data.frame()
19 for(k in sc){cost=carbon=0;peak=0;vari=0
20 for(r in R){q=rt[rt$区域编号==r,];s=st[st$区域编号==r,]

```

```

21 fac=(q$基准人工智能信息技术负荷+q$非人工智能信息技术负荷)*pue[r];net=fac-q$直接消纳新能源
22 pg=stor(net,price[,r],s$储能容量*k,s$最小荷电状态*k,s$初始荷电状态*k,s$最大充电功率,s$最大放电功率,s$充电效率,s$放电效率,s$外送上限
    )
23 cost=cost+sum(pg*price[,r]);carbon=carbon+sum(pg*ci[,r]);peak=max(peak,max(pg));vari=vari+sd(pg)}
24 res=rbind(res,data.frame(容量倍数=k,购电成本万元=round(cost/1e4,1),碳排放吨=round(carbon,0),峰值净购电_MW=round(peak,1),负荷波动=
    round(vari,2)))}
25 write_csv(res,file.path(od,"灵敏度_储能容量.csv"));print(res)
26 cat("输出目录:",od,"\n")

```

灵敏度分析 _ 新能源波动.R

```

1 library(readr);library(dplyr)
2 set.seed(1)
3 od=file.path("C:/Users/18904/Github/huashu-cup","解答","5-灵敏度分析");ap=file.path("C:/Users/18904/Github/huashu-cup","题目","附件数据")
4 rt=read_csv(file.path(ap,"region_time_data_region_time_data.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
5 st=read_csv(file.path(ap,"storage_information_storage_information.csv"),locale=locale(encoding="UTF-8"),show_col_types=FALSE)
6 pue=c(区域A=1.35,区域B=1.35,区域C=1.38,区域D=1.28,区域E=1.25,区域F=1.27)
7 R=st$区域编号;TT=2407
8 price=ci=matrix(0,TT,6);colnames(price)=colnames(ci)=R
9 for(j in seq_along(R)){s=rt[rt$区域编号==R[j],];price[,j]=s$购电价格;ci[,j]=s$碳强度}
10 stor=function(net,price,E,Smin,S0,Pc,Pd,ec,ed,Smax){win=24;tg=as.numeric(stats::filter(net,rep(1/win,win),sides=2));tg[is.na(tg)]=net[is.na(tg)]
11 band=sd(net)*.2;q=quantile(price,c(.35,.65));soc=S0;n=length(net);pg=ds=ch=numeric(n)
12 for(t in 1:n){if(net[t]<0){sc=min(Pc,-net[t]/(E-soc)/ec);soc=soc+ec*sc;pg[t]=0}
13 else if(net[t]>tg[t]+band&&price[t]>q[1]&&soc>Smin){ds[t]=min(Pd,net[t]-tg[t]/(soc-Smin)*ed);soc=soc-ds[t]/ed;pg[t]=net[t]-ds[t]}
14 else if(net[t]<tg[t]-band&&price[t]<q[2]&&soc<E){gc=min(Pc,tg[t]-net[t]/(E-soc)/ec);soc=soc+ec*gc;pg[t]=net[t]+gc}
15 else{pg[t]=net[t]}}
16 if(soc<S0)for(t in n:1){if(soc>=S0)break;gc=min(Pc,(S0-soc)/ec,(E-soc)/ec);if(gc<=0)next;soc=soc+ec*gc;pg[t]=pg[t]+gc}
17 pg}
18 sc=c(1,.9,.8,.7,.6);res=data.frame()
19 for(k in sc){cost=carbon=0;peak=0;vari=0
20 for(r in R){q=rt[rt$区域编号==r,];s=st[st$区域编号==r,]
21 fac=(q$基准人工智能信息技术负荷+q$非人工智能信息技术负荷)*pue[r];net=fac-q$直接消纳新能源*k
22 pg=stor(net,price[,r],s$储能容量,s$最小荷电状态,s$初始荷电状态,s$最大充电功率,s$最大放电功率,s$充电效率,s$放电效率,s$外送上限)
23 cost=cost+sum(pg*price[,r]);carbon=carbon+sum(pg*ci[,r]);peak=max(peak,max(pg));vari=vari+sd(pg)}
24 res=rbind(res,data.frame(新能源保留=k,购电成本万元=round(cost/1e4,1),碳排放吨=round(carbon,0),峰值净购电_MW=round(peak,1),负荷波动=
    round(vari,2)))}
25 write_csv(res,file.path(od,"灵敏度_新能源波动.csv"));print(res)
26 cat("输出目录:",od,"\n")

```